

Masterarbeit

Exploring Efficient Autoregressive Diffusion in Different Representation Spaces

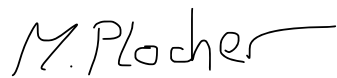
Eberhard Karls Universität Tübingen
Mathematisch-Naturwissenschaftliche Fakultät
Wilhelm-Schickard-Institut für Informatik
Lernbasierte Computer Vision
Marcel Plocher, marcel.plocher@student.uni-tuebingen.de, 2026

Bearbeitungszeitraum: 01.12.2025-01.06.2026

Betreuer/Gutachter: Prof. Dr. Andreas Geiger, Universität Tübingen
Zweitgutachter: Prof. Dr. Georg Martius, Universität Tübingen

Selbstständigkeitserklärung

Hiermit versichere ich, dass ich die vorliegende Masterarbeit selbständig und nur mit den angegebenen Hilfsmitteln angefertigt habe und dass alle Stellen, die dem Wortlaut oder dem Sinne nach anderen Werken entnommen sind, durch Angaben von Quellen als Entlehnung kenntlich gemacht worden sind. Diese Masterarbeit wurde in gleicher oder ähnlicher Form in keinem anderen Studiengang als Prüfungsleistung vorgelegt.



Marcel Plocher (Matrikelnummer 6519595), June 1, 2026

Abstract

Modern generative models are trained on vast data and require extensive computational resources. In joint diffusion models, which capture the joint distribution of all image tokens simultaneously, incorporating strong semantic representations has been shown to significantly speed up convergence. In contrast, continuous masked autoregressive models capture token interdependencies via autoregression, jointly with the per-token distribution through diffusion. However, previous work in this paradigm has relied entirely on the compressed Stable Diffusion Variational Autoencoder (SD-VAE). To address this, we introduce a generalized pipeline that accommodates different target representation spaces and models the per-token distribution based on rectified-flow. Our experiments demonstrate that leveraging semantically meaningful Representation Autoencoders (RAEs) enables our model to converge substantially faster than our baseline utilizing SD-VAE. Furthermore, we show that operating directly in pixel space, which eliminates the need for a pre-trained tokenizer, is viable but significantly less efficient during training. Ultimately, our results indicate that synthesizing masked autoregressive modeling with semantically meaningful target representations offers a highly promising pathway toward resource-efficient generative modeling.

Acknowledgments

First, I would like to express my gratitude to Prof. Andreas Geiger for providing me with the opportunity to work on this interesting research project within his group. I am equally grateful to Gege Gao, whose exceptional guidance, feedback, and technical insights steered me through every stage of this work and deeply shaped my growth as a researcher.

Furthermore, I would like to thank Merve Kocabas and Bastian Berle for the helpful technical discussions and meaningful personal conversations during my time working on this thesis.

On a personal note, I want to extend a special thanks to my parents, who have always been there for me and whose advices helped me navigate important life decisions. Finally, I would like to thank Anna for always listening and for providing the best distractions whenever I needed them.

Contents

1. Introduction	11
2. Related Work	15
2.1. Diffusion and Flow Models	15
2.2. Representations and Image Generation	16
2.3. Sequential Image Generation	17
2.4. Evaluation Metrics for Image Generation	18
3. Analysis and Definition of Masked Autoregressive Flow Matching	21
3.1. Model Definition	21
3.1.1. Per Token Flow Matching Objective	22
3.1.2. Masked Autoregression with Continuous-valued Tokens	24
3.2. Model Architecture	24
3.3. Trajectory Distribution Sampling	26
3.4. Generation Guidance	29
3.5. Analysis of Different Representation Spaces	30
3.5.1. Definition of Representation Spaces	30
3.5.2. Metrics for Representation Spaces	31
3.5.3. Masking in Different Spaces	34
3.5.4. Assumptions for Experiments	36
4. Model Implementation	37
4.1. Implementation of the Transformer Backbone	37
4.2. Implementation of the Denoiser	39
4.3. Model Parameters and Training Details	40
5. Experiments on Different Spaces	41
5.1. Baseline	41
5.2. RAE Latent Space	44
5.3. Pixel Space	48
5.4. Comparison of Different Representation Spaces	51
5.4.1. Training Efficiency	51
5.4.2. Final Generation Quality	51
5.4.3. Masking during Training	53
5.4.4. Inference Parameters	55
5.4.5. Visualization of the Autoregressive Generation Process	56

Contents

6. Discussion	59
6.1. Limitations	59
6.2. Further Research and Directions	60
6.3. Conclusion	61
A. Additional Implementation Details	71
A.1. Details for Representation Autoencoders (RAE)	71
A.2. Detailed Parameters and Architectures	72
B. Qualitative Results	75

1. Introduction

Generative models are typically trained using large amounts of data, a process that demands substantial computational resources. Consequently, the training of modern, large-scale foundational models is often restricted to commercial entities or well-funded academic laboratories. A significant milestone toward computationally efficient generative modeling was achieved through the introduction of Stable Diffusion Variational Autoencoder (SD-VAE) from [1]. This type of continuous tokenizers leverage spatial and channel-wise compression together with a pixel-reconstruction objective to produce low-capacity latent representations that effectively capture local appearance.

However, training efficiency can be dramatically improved by utilizing semantic meaningful vision foundation models, such as [2, 3, 4]. Building on this, [5] successfully trained decoders to map these high-level features back to the pixel space. Such Representation Autoencoders (RAEs) construct a semantically meaningful representation space, enabling both efficient and effective training of diffusion and flow-matching models

Another approach to enhancing the efficiency of generative frameworks is introduced by [6]. Their contribution is that under-capacity networks can generate images effectively within high-dimensional spaces by configuring the neural network to directly predict the clean data trajectory (x-prediction). The authors demonstrate that clean, natural data inherently lies on a low-dimensional manifold, whereas noise quantities do not.

Both explained approaches were originally explored within the context of joint flow-matching models, where all image tokens are generated simultaneously. Yet, it remains unexplored whether masked autoregressive models with continuous-valued tokens [7, 8] can similarly benefit from these approaches, and whether they can successfully leverage the same underlying performance advantages.

Joint flow-matching aims to learn a global velocity field v_Θ to model the entire image distribution $p(x)$ simultaneously, where $x \in \mathbb{R}^{N \times D}$ with N is the sequence length.

In contrast, masked autoregressive models factorizes the image distribution $p(x)$ into a product of conditional distributions via the chain rule:

$$p(x) = p(x_1, \dots, x_n) = \prod_{i=1}^n p(x_i | x_1, \dots, x_{i-1}), \quad (1.1)$$

where x_i denotes a single token. Under this factorization, we can model the token interdependence by autoregression and the per-token distribution by flow-matching using a velocity field $\hat{v}_\Theta$. This formulation transforms the global generation task into a conditional flow-matching problem localized to a single target token x_i given a context vector z_i from a autoregressive transformer backbone.

Another distinct advantage of masked autoregressive models is that the encoder component of the transformer backbone operates exclusively on unmasked tokens. This characteristic substantially reduces the computational overhead of the self-attention mechanism, which scales as $O(N^2)$, where N denotes the number of tokens.

While previous work on masked generative models [7, 8] has relied on the compressed SD-VAE [1] latent space, we aim to explore alternative representation spaces. In our research, we systematically address the following core questions:

1. Can we train continuous masked generative models more efficiently by identifying an optimal representation space.
2. Can we leverage masked autoregressive structures to enhance the training efficiency of generative models.

In our work we use the well known SD-VAE latent space as a baseline. We further explore within RAEs, specifically we leverage DINOv2 [3] and Masked Autoencoder (MAE) [4]. In contrast, we also evaluate within pixel space that bypasses tokenization entirely. In Fig. 1.1 we visualize the features of a encoded image for each selected representation space. In this work we will analyze these diverse representation spaces using suitable metrics to make a assumption before our experiments.

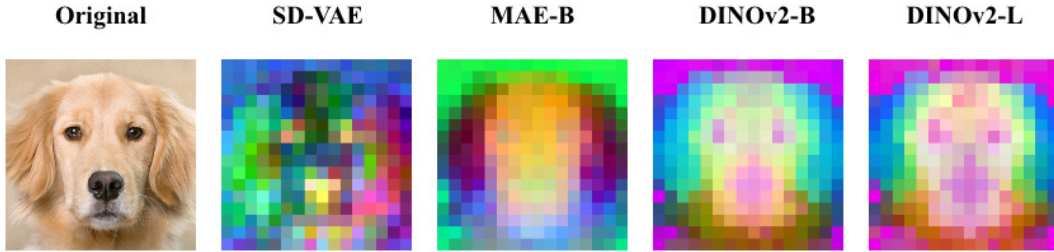


Figure 1.1.: Visual comparison of the selected representation spaces. We use Principal Component Analysis (PCA) coloring to visualize the features of each representation.

We begin our experiments by establishing a baseline with SD-VAE. We further optimize this baseline by replacing the standard uniform timestep distribution with the logit-normal sampling schedule introduced by [9]. Experimentally, we demonstrate that incorporating a progressive Classifier-Free Guidance (CFG) schedule yields the optimal trade-off between structural fidelity and sample diversity, as quantified by the Fréchet Inception Distance (FID) metric.

Given this baseline, we experiment with utilizing high-dimensional, semantically rich RAEs. We begin by evaluating the DINOv2 latent space while maintaining the hyperparameter and architecture as for our baseline. We systematically optimize the model by iteratively adjusting various hyperparameters and structural components. Interestingly, we observe that CFG fails to improve the generative performance when operating within the DINOv2 representation space. In addition, we evaluate using MAE as latent space, whose underlying masking and reconstruction objective differs fundamentally from the properties of DINOv2. Aligning with the observations made by [5], we find that the MAE latent space yields poor generation performance.

We further experiment within the pixel space, training our generative model entirely end-to-end without the aid of a tokenizer. Through this setup, we investigate whether a latent space is necessary for masked flow matching models. Following the methodology of Just Image Transformer (JiT) [6], we evaluate manifold prediction (x-prediction) for our conditional, per-token distribution. Our empirical results indicate that while predicting the clean data trajectory proves beneficial, it does not alleviate the high-dimensionality as effectively as it does within joint flow-matching. Our experiment reveal that CFG is absolutely critical for achieving high-fidelity image generations when operating directly within the pixel space.

Finally, we compare all representation spaces in terms of training efficiency. Our empirical results demonstrate clearly that RAEs, utilizing a DINOv2 latent space, can be trained most efficiently. Notably, our final generation quality using DINOv2 is lower than SD-VAE with its best-performing guidance scale. In contrast, training the generative model end-to-end within the pixel space proves to be the least efficient approach.

2. Related Work

This chapter provides a overview of the theoretical foundations and recent advances related to our work. We begin by providing the formulations of diffusion and flow matching, which defines learnable continuous trajectories that map between arbitrary distributions. Building on these generative principles, we introduce Diffusion Transformers (DiTs) as well as latent diffusion models. We then discuss how representation learning can be tightly integrated with generative modeling to enhance training efficiency and structural fidelity. Next, we review sequential architectures, exploring discrete autoregressive approaches, masked generative models, and their corresponding loss formulations. Finally, we will go over recent evaluation metrics for generative models.

2.1. Diffusion and Flow Models

Diffusion and Flow Schemas. The foundational formulation of diffusion for generative models was introduced in the pioneering work of [10]. Their approach aimed to learn a stochastic reverse process that stepwise removes noise to recover the original data distribution. Subsequent works [11, 12, 13] significantly popularized this paradigm. Specifically, Denoising Diffusion Probabilistic Model (DDPM) [11] formulated the stochastic reverse process by training a network to predict the added noise via ϵ -prediction. To improve inference efficiency, Denoising Diffusion Implicit Model (DDIM) [12] introduced a deterministic sampling mechanism that requires substantially fewer sampling steps. Furthermore, [13] demonstrated that for every Stochastic Differential Equation (SDE) path, there exists a corresponding deterministic Ordinary Differential Equation (ODE) that yields the exact same marginal probability distribution. More recently, a simpler and more modern formulation has emerged in the form of flow matching models [14, 15], where the neural network directly predicts the velocity field between the noise and the clean data. Finally, [9] demonstrated that diffusion and flow-matching models can be viewed through a single, unified framework and introduced the logit-normal distribution for timestep sampling.

Diffusion Transformer. The transformer architecture, originally proposed by [16], has largely replaced domain-specific architectures across language modeling [17], computer vision [18], and numerous other fields in machine learning. More recently, the transformer architecture has also emerged as the dominant backbone

for diffusion-based generative models. A pioneering work in this direction, DiT [19], systematically explored the scaling behaviour of transformer architectures within the diffusion paradigm. This approach was subsequently refined for enhanced computational efficiency and higher generative performance in Scalable Interpolant Transformer (SiT) [20].

Latent Diffusion Models. A breakthrough in image generation was the introduction of Latent Diffusion Models (LDMs) [1] where a pre-trained continuous autoencoder is used to compress the raw pixels by removing imperceptible details. This compressed latent space is known to let diffusion and flow models train efficient and still keeps the high-resolution generation quality. In [19, 7, 20, 21] they utilized this latent space. Recent works explored also other latent spaces such as RAE [5, 22] and end-to-end image generation models in pixel space [6, 23, 24, 25].

2.2. Representations and Image Generation

Representation Learning. The core objective of representation learning is to extract semantic and disentangled representations that simplify subsequent downstream tasks. To achieve this, diverse pre-training objectives have been developed. One prominent paradigm relies on contrastive learning, which aims to maximize the agreement between different augmented views of the same image [26, 27]. Alternatively, self-distillation methods such as the DINO family [2, 3, 28] utilize a student network tasked with learning from a momentum-updated teacher network. Another field of research [29, 30] focuses on aligning language and vision in one shared embedding space, which enables zero-shot classification. Conversely, MAEs [4] adopt a generative reconstruction approach, where a substantial portion of the input image is masked and a decoder is trained to reconstruct the missing patches.

Representation Alignment for Image Generation. Emerging research directions [31, 32, 33] focus on aligning image generation with representation learning to achieve more meaningful and context-aware synthesis. By integrating representations, the generative model develops a stronger awareness of the generation target. Furthermore, aligning image generation with semantic representations enhances semantic consistency and accelerates convergence, thereby improving overall training efficiency. A pioneering work in this area, Representation Alignment (REPA) [31], utilizes a frozen DINOv2 encoder [3] to explicitly align the intermediate layers of a diffusion transformer. Conversely, Self-Flow [32] avoids reliance on external representations by introducing an entirely self-supervised approach that incorporates an auxiliary representation loss within a student-teacher framework. Their findings demonstrate that representation learning and generative modeling can be unified within a single architecture without external alignment, offering superior scaling behaviour and strong generalization across modalities.

Representation Spaces. In [5], the authors trained decoders for three distinct representation encoders (DINOv2 [2], MAE [4], SigLib [30]) to reconstruct pixels directly from their respective feature spaces, a framework they termed RAEs. Leveraging the strong semantic grounding of these representation encoders, they observed accelerated convergence, leading to significantly enhanced training efficiency alongside State-of-the-Art (SOTA) generation quality. However, the reconstruction fidelity of this RAE latent space remains limited compared to traditional autoencoders explicitly trained with a reconstruction loss [34]. To address similar trade-offs, [35] proposed training a Variational Autoencoder (VAE) that is explicitly aligned with vision foundation models.

2.3. Sequential Image Generation

Sequential Models. Sequential image generative models decompose the generation task into a sequence of conditional predictions by relying on previously generated context. Pioneering work in this domain was introduced by [36, 37]. While PixelCNN [36] relied on convolutional architectures, ImageGPT [37] introduced the modern transformer architecture to autoregressive modeling.

Autoregressive Image Generation with Discrete Tokens. A line of research [38, 39] models images as sequences of discrete tokens via vector quantization. This approach was heavily motivated by the success of autoregressive language models, which are known to scale well. Building on classical "next-token prediction", [40] systematically explored and improved the scaling behaviour of these models for visual synthesis. Additionally, a distinct paradigm within sequential image generation was introduced by [41], which utilizes a coarse-to-fine next-scale prediction mechanism rather than traditional raster-scan token prediction.

Masked Image Generation. In contrast to purely autoregressive approaches, another prominent line of research focuses on masked generative models utilizing discrete tokens [42, 43]. Specifically, [42] demonstrated that treating images strictly as a linear sequence of tokens is neither optimal nor computationally efficient. Instead, they proposed a transformer decoder architecture equipped with bidirectional attention. During training, the decoder learns to predict randomly masked tokens by attending to unmasked contexts in all spatial directions. Expanding upon masked generative frameworks, Masked Generative Encoder (MAGE) [43] unified representation learning and image generation within a single model architecture.

Masked Image Generation with Continuous Tokens. In Masked Autoregressive (MAR) [7], the authors combined the framework of masked generative modeling with a continuous diffusion loss to model images autoregressively. While utilizing bidirectional attention to implement mask autoregression similar to MaskGIT [42], they introduced a diffusion head to explicitly model the conditional distribution $p(x_i | z_i)$ for each token i , where z_i represents the contextual conditioning vector

emitted by the bidirectional transformer backbone. Within this setup, the diffusion head essentially acts as a localized loss function for the quality of z_i . Subsequently, [44] analysed the scaling behaviour of autoregressive models in the context of text-to-image generation, observing that sequential models operating on continuous tokens and bidirectional attention models with random masking exhibit the most favorable scaling trajectories. Further refining this approach, [8] investigated various mask-ratio schedules during training to optimize the generative efficiency of the MAR framework. Alternatively, [45] explored causal autoregression utilizing an attention mask paired with a continuous loss. To mitigate the high computational complexity arising from a combined transformer architecture that simultaneously handles both autoregression and diffusion, they proposed grouping token cells to maintain a computationally tractable generation process.

2.4. Evaluation Metrics for Image Generation

Inception Score. The Inception Score (IS) was introduced by [46] as the first widely adopted metric for evaluating generative image quality. It explicitly quantifies two distinct properties: (i) object quality, meaning individual images should contain clearly identifiable objects, and (ii) diversity, meaning the model should generate a wide variety of object classes. The metric leverages a pre-trained Inception-v3 network [47] to compute the conditional class distribution $p(y | x)$ for a generated image x given class label y . The final score is determined by calculating the Kullback-Leibler (KL) divergence between this conditional distribution and the marginal class distribution $p(y)$. A high-quality image yields a low-entropy conditional distribution (indicating high classification certainty), while a diverse model yields a high-entropy marginal distribution (indicating an even distribution across all classes). A major limitation of the IS is its intra-class diversity, meaning a model could generate identical images within a single class without penalty. Furthermore, the metric relies solely on classifier outputs and lacks a direct comparison to real reference images.

Fréchet Inception Distance (FID). While the IS marked a significant breakthrough in evaluating generative performance, it suffers from a fundamental limitation: it does not directly compare generated samples against real reference data. Currently, the most prevalent metric to address this is the FID, introduced by [48]. Instead of relying on classifier probabilities, FID extracts intermediate feature vectors from the pre-trained Inception-v3 network [47] for both real and generated image sets. These high-dimensional features are then modeled as multivariate Gaussian distributions. The FID is subsequently computed as the distance (specifically, the Wasserstein-2 distance) between the estimated Gaussian distributions of the real data and the generated data.

Precision and Recall. When evaluating generative performance, visual synthesis is primarily characterized by two core dimensions: fidelity (perceptual quality) and diversity (coverage of the data distribution). While metrics like FID and IS combine

these aspects into a single scalar value, the precision and recall metrics introduced by [49] explicitly disentangles them. Their approach utilizes a k-nearest neighbor (k-NN) algorithm to construct local hyperspheres around each data point in an embedding space. Precision is defined by the proportion of generated samples that fall within the neighbourhood of at least one real data point, serving as a metric for fidelity. Conversely, recall is defined by the proportion of real data points that fall within the neighbourhood of at least one generated sample, quantifying diversity coverage. This explicit separation makes the precision and recall metric highly effective for diagnosing specific failure modes in generative models, such as mode collapse (low recall) or the synthesis of blurry and distorted artifacts (low precision).

LPIPS Reconstruction Quality. A widely adopted metric for evaluating the reconstruction quality and perceptual fidelity of images is the Learned Perceptual Image Patch Similarity (LPIPS) [50]. Instead of relying on traditional pixel-level comparisons LPIPS directly compares two images by passing both through an ImageNet pre-trained convolutional network backbone, such as VGG [51], AlexNet [52], or SqueezeNet [53]. Then, features are extracted from several intermediate layers, normalized, and compared using learned linear distances.[50] demonstrated that features of Convolutional Neural Networks (CNNs) inherently capture edges, textures, and object parts in a manner that aligns closely with human visual judgments.

3. Analysis and Definition of Masked Autoregressive Flow Matching

We want to start our research with a analysis and definitions for masked flow matching models. Our work builds upon MAR [7] and we want to generalized this models for using other target representations. We start with the definition of a continuous loss-function and a sampler using per-token flow-matching. We then use this to define the autoregressive modeling using continuous tokens. Further we explain our model pipeline during training and inference. Finally, we will analyze different representations and compare them.

3.1. Model Definition

In MAR the authors revisit the roles of discrete-valued tokens in autoregressive generation models. They found that using continuous-valued tokens outperforms vector-quantized tokens. Therefore they defined a per-token diffusion loss following DDPM [11]. They modeled the interdependence of tokens by autoregression jointly and the per-token distribution with diffusion.

We follow MAR and also present the role of discrete-valued tokens in autoregressive generation models. Let x_i be the token to predict in the next autoregressive step. We can represent x_i by using a scaler $0 < x_i < K$ with a vocabulary size K . We can train a autoregressive model that generates a continuous vector $z_i \in \mathbb{R}^D$ where D is the dimension of the token. We can use a k-way classifier $W \in \mathbb{R}^{K \times D}$ to project z_i into the vocabulary K . So autoregressive models use a categorical probability distribution in the form $p(x_i|z_i) = \text{softmax}(Wz_i)$. We can define the loss-function with a simple cross-entropy loss for the categorical distribution. During sampling we then draw from $p(x_i|z_i) = \text{softmax}(Wz_i/\tau)$ where τ is the temperature that controls the diversity. To replace the discrete-valued with continuous-valued tokens we need to exhibit the following two properties:

- A **loss-function** to measure the difference between the predicted and true distribution.
- A **sampler** that draws samples from the distribution $x \sim p(x|z)$.

Therefore is not necessary to use discrete tokens for autoregressive models. It is more principle to model a distribution that is essential and suitable. In the next section we

therefore define a loss-function and a sampler.

3.1.1. Per Token Flow Matching Objective

Joint Flow Matching. Flow matching models learn the continuous-time probability path from a simple noise distribution to the data distribution. Let $x_0 \in \mathbb{R}^{N \times D}$ be the clean data with the sequence of N tokens and a channel dimension of D . Following [14] we can define a probability path by interpolation between a noise distribution $x_1 \sim \mathcal{N}(0, I)$ and the data distribution $x_0 \sim p_{data}$ with:

$$x_t = \alpha(t)x_0 + \sigma(t)x_1, \quad (3.1)$$

where $\alpha(t)$ and $\sigma(t)$ are the interpolation parameters for data and noise. In our work we specifically use rectified flow [15, 9] which builds a linear straight-line trajectory with $\alpha(t) = 1 - t, \sigma(t) = t$. This results then in an interpolation:

$$x_t = (1 - t)x + t\epsilon. \quad (3.2)$$

Here, the parameter $t \in [0, 1]$ is the timestep where $t = 1$ corresponds to pure noise and $t = 0$ corresponds to clean data. The target velocity field along this path is defined by:

$$v_t = \frac{dx_t}{dt} = \epsilon - x. \quad (3.3)$$

We can train a neural network $v_\Theta(x_t, t)$ to learn the target velocity field v_t by minimizing the following objective:

$$\mathcal{L}_{\text{gen}} = \mathbb{E}_{t, x_0, \epsilon} \|v_\Theta(x_t, t) - v_t\|^2. \quad (3.4)$$

Per Token Flow Matching. As we want to utilize the benefits of masked generative models we need to define a velocity field for each token. The autoregressive model predicts use a global condition $z \in \mathbb{R}^{B \times N \times D}$. Given the context z , we denoise every token x_i , where i denotes the index, separated. During training we can run all tokens in parallel by reshaping z and x to the shape $[B \cdot N, C]$. We random sample the timestep $t_i \sim [0, 1]$ for each token separately. Doing so we get the following per token interpolation:

$$x_{t,i} = (1 - t_i)x_i + t_i\epsilon_i, \quad (3.5)$$

where again $t_i = 0$ denotes clean data and $t_i = 1$ pure noise. Similar to joint flow-matching we define our per token target with:

$$v_{t,i} = \frac{dx_{t,i}}{dt_i} = \epsilon_i - x_i. \quad (3.6)$$

We follow MAR and only use the masked tokens $i \in M$ for our final objective:

$$\mathcal{L}_{\text{gen}} = \mathbb{E}_{t,x_i,\epsilon} \left[\frac{1}{|M| \cdot D} \sum_{i \in M} \|v_{\Theta}(x_{t,i}, z_i, t_i) - v_{t,i}\|^2 \right], \quad (3.7)$$

where the neural network v_{Θ} is also parametrized by the global condition z . We normalize the loss $\mathcal{L}_{\text{gen}}$ by the number of masked tokens $|M|$ and the channel-dimension D .

Manifold Prediction. In JiT [6] the authors found that predicting the clean data x_0 (x-prediction) allows a neural network with limited capacity to handle much higher-dimensional data. In our research we similarly want to explore if x-prediction can be beneficial. To apply x-prediction we just need to parameterize the output of the neural network with:

$$x_{\Theta} = \text{net}_{\Theta}(x_t, t). \quad (3.8)$$

We then need to transform the x-prediction output to a velocity with:

$$v_{\Theta} = \frac{x_{\Theta} - x_t}{1 - t}. \quad (3.9)$$

As the velocity target from Eqn. 3.3 is off-manifold, we need to define the target with:

$$v_t = \frac{x_t - \epsilon}{t}. \quad (3.10)$$

Per-token Sampler. At inference time we want to sample from the distribution $p(x_i|z_i)$ where i denotes the i -th token. From [15] it is known that a deterministic ODE sampler is the most effective and efficient sampling method for rectified flow. Therefore at inference we define the ODE with:

$$\frac{dx_{t,i}}{dt} = v_{\Theta}(x_{t,i}, z_i, t_i), \quad (3.11)$$

where we start from pure noise $x_t \sim \mathcal{N}(0, I)$ for $t = 1$ backwards in time until $t = 0$ to get samples from the learned data distribution.

3.1.2. Masked Autoregression with Continuous-valued Tokens

Next-token Prediction. We now want to formulate the autoregressive "next token prediction" for our model. Given a sequence of tokens $x = \{x_1, x_2, \dots, x_n\}$ where the index $1 \leq j \leq n$ specifies an order. We can use a neural network f_Θ to represent the conditional distribution:

$$p(x) = p(x_1, \dots, x_n) = \prod_{j=1}^n p(x_j | x_1, \dots, x_{j-1}). \quad (3.12)$$

We can also rewrite the conditional distribution $p(x_i | x_1, \dots, x_{i-1})$ into two parts. (i) A conditional vector $z_i = f_\Theta(x_1, \dots, x_{i-1})$ where f_Θ is a neural network operating on the full token sequence. (ii) A per-token probability distribution $p(x_i | z_i)$ which we model with our defined per-token flow matching loss in Eqn. 3.7. Then we can use backpropagation to z_i for updating the parameters of f_Θ .

Mask Autoregressive Models. In MAR they unified the definition of autoregression and mask generative models [42, 43] where mask autoregression is defined with a random order autoregression. This can be done with a transformer architecture like [18] and bidirectional attention instead of common causal attention. In mask autoregression it is possible to predict a random subset of tokens in parallel. Our conditional distribution can then be formulated with:

$$p(\{x_i, x_{i+1}, \dots, x_j\} | x_1, \dots, x_{i-1}), \quad (3.13)$$

where multiple tokens $\{x_i, x_{i+1}, \dots, x_j\}$ are predicted in parallel and $(i \leq j)$.

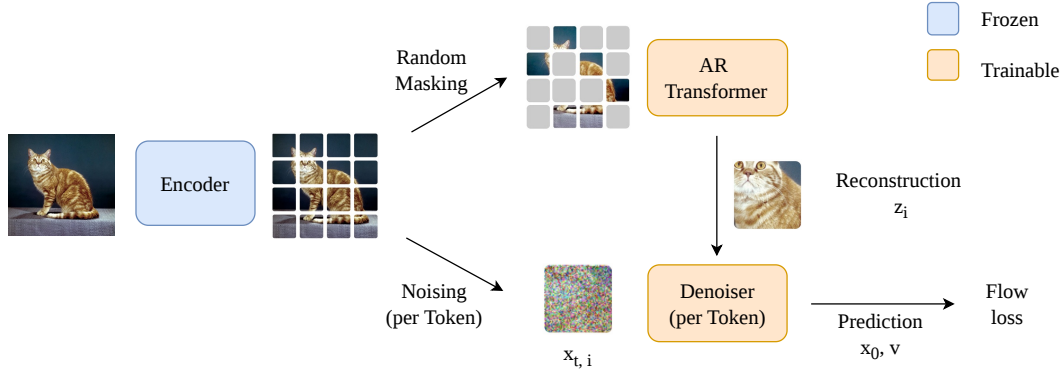
3.2. Model Architecture

In this section, we present our model architecture, which builds upon the structure foundations of MAR, but is formulated more generally to accommodate alternative representation spaces. Because the architecture differs between training and inference, we detail these phases separately. While this section provides a high-level architectural overview, more far-reaching implementation details regarding specific network configurations are provided in Chapter 4.

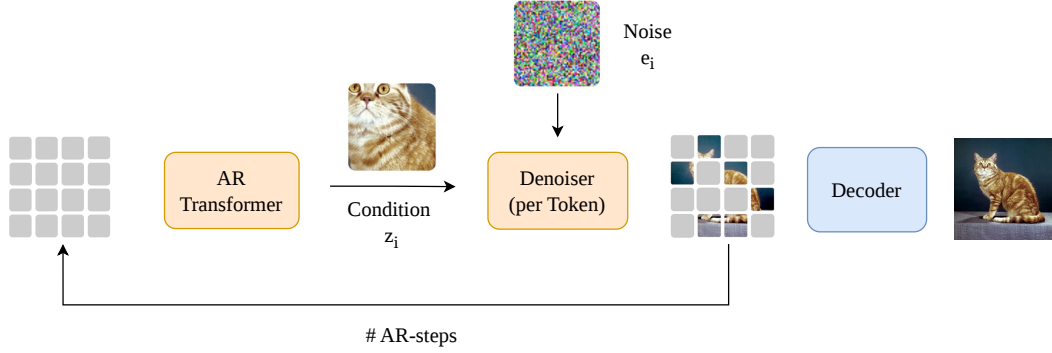
Training Pipeline

Our overall architectural layout during training is illustrated in Fig. 3.1a. The pipeline utilizes an frozen encoder to map images into latent tokens, establishing the underlying continuous space where the flow-matching objective are optimized. Notably, when treating raw pixel space as a direct representation, the encoder phase

is bypassed, and a patchification is instead used to extract tokens directly from the input image.



- (a) **Training Pipeline.** Images are mapped into a latent token space (or patchified directly in pixel space). A random subset of these tokens is masked out. The bidirectional transformer predicts the global context z from the unmasked sequence, while the per-token denoiser learns to predict the clean target from noised tokens while relying on z .



- (b) **Inference Pipeline.** Beginning from a fully masked token grid, the model iteratively selects a subset of unmasked tokens at each step. The bidirectional transformer computes the global context z , which serves as a conditions for the denoiser to generate the next set of clean tokens over a predefined autoregressive schedule.

Figure 3.1.: Architectural pipeline for masked autoregressive flow matching across both training and inference.

In the transformer backbone of our masked flow matching model, a random selected portion of the token sequence is masked out, and the model is optimized to reconstruct these hidden tokens from the remaining unmasked sequence. Through this objective, the network learns a global contextual embedding that serves as the conditioning signal for the per-token denoiser.

The secondary component of our framework is the localized, per-token denoiser, which models the conditional token distribution $p(x_i | z_i)$. During the training phase, Gaussian noise is linearly interpolated into the ground-truth latent tokens according

to our flow schedule (see Eqn. 3.5). Given these corrupted data token, the denoiser is conditioned on z_i to predict the clean token target, or its corresponding velocity field and is optimized via our formulated flow loss in Eqn. 3.7.

Inference Pipeline

During inference, we begin the generation process by initializing all tokens to zero. In the initial autoregressive step, the model relies entirely on the localized denoiser and the corresponding class information. For all subsequent steps, the newly generated tokens are passed into the contextual transformer network to reconstruct the global context z (see Fig. 3.1b). Given this updated global context, the per-token denoiser integrates the active target tokens, transforming them from pure Gaussian noise into clean data using context z as condition. This sequence is executed autoregressively according to a predefined order until every token across the grid has been fully denoised. Finally, because the entire generative process operates within a latent space, the fully assembled sequence is passed through the corresponding decoder to map the latent representation back into the raw pixel space.

3.3. Trajectory Distribution Sampling

In our defined model pipeline, we have two distinct and crucial distributions required for sampling. We begin by defining the sampling strategy for the continuous flow timesteps, followed by the mask-rate distribution.

Timestep Sampling

Recent studies [9, 32, 34, 5, 54] have demonstrated that the distribution for sampling timestep t in rectified flow matching [15] is a critical determinant of both generative performance and training efficiency. Consequently, we hypothesize that optimization of this distribution is important within our proposed model. In the following sections, we formally define the specific timestep distributions.

Uniform Sampling. The baseline for our experiments will be the uniform timesampling $t \sim \mathcal{U}(0, 1)$. This distribution is applied the pioneer works of rectified flow-matching [15, 20].

Logit-Normal Distribution. In [9] they formulated the logit-normal distribution [55] for the timestep sampling. This distribution can be obtained by "squashing" a normal distribution, which has the range of $[-\infty, \infty]$, into $[0, 1]$. The density is defined as:

$$p_m(t, \mu, \sigma) = \frac{1}{\sigma \sqrt{2\pi}} \frac{1}{t(1-t)} \exp\left(-\frac{(\text{logit}(t) - \mu)^2}{2\sigma^2}\right), \quad (3.14)$$

3.3. Trajectory Distribution Sampling

where μ is the location and σ the scale parameter. The $\text{logit}(t)$ is defined as $\text{logit}(t) = \log \frac{t}{1-t}$.

For the practical use of this distribution we can simply define it in two steps. First sample from a normal distribution:

$$u \sim \mathcal{N}(\mu, \sigma) \quad (3.15)$$

and then transform u into t by using the Sigmoid function:

$$t = \frac{1}{1 + e^{-u}}. \quad (3.16)$$

Shift Function. Recent work [9, 32, 34, 5, 54] has explored that the generation quality and training efficiency can be significantly improved by shifting the timestep distribution. A shift can be applied into the direction of clean data ($t = 0$) or the direction of pure noise ($t = 1$). In [9] they defined a timeshift shift as:

$$s(\alpha, t) = \frac{\alpha t}{1 + (\alpha - 1)t}, \quad (3.17)$$

where α is the shifting parameter. We can apply this shift by sample from the logit-normal distribution $t \sim p_{ln}$ and pass it through $s(\alpha, t)$. This changes the location parameter to $\mu' = \mu + \log(\alpha)$. In Fig. 3.2 we plotted the logit-normal distribution with different shift parameters α .

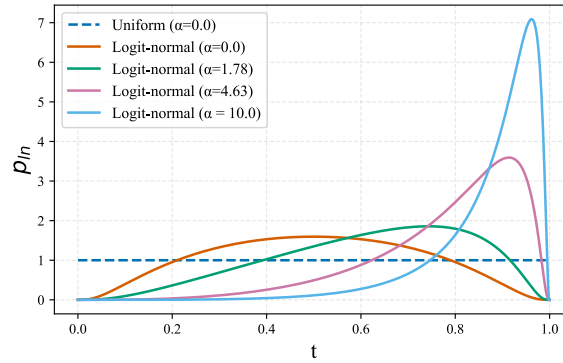


Figure 3.2.: Probability Density Functions (PDFs) of the logit-normal distribution under varying time-shift parameters α . Adjusting α systematically shifts the sampling density along the trajectory between clean data states ($t = 0$) and noise states ($t = 1$).

Specifically, [9] and [34] formalize a logit-normal timestep distribution modified by a train-shift parameter α_{train} and a sample-shift parameter α_{sample} . Adjusting

the sample-shift such that $\alpha_{\text{sample}} > 1.0$ during inference systematically shifts the ordinary differential equation (ODE) integration timesteps toward the high-noise regime. As a consequence, a greater proportion of the computational budget during sampling is allocated to resolving the early noisy timesteps of the data distribution.

Mask Rate Sampling

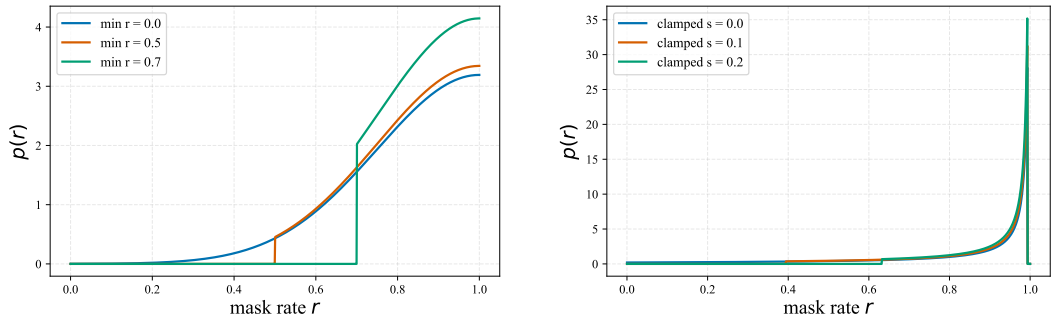
The mask-rate parameter directly regulates the proportion of tokens masked during training. A high mask-rate increases task complexity for the transformer decoder by restricting available context. In MAR, the mask-rate density is modeled as a truncated normal distribution restricted to the interval $[\text{mask}_{\min}, 1.0]$, as illustrated in Fig. 3.3a. The authors empirically fixed $\text{mask}_{\min} = 0.7$, biasing training toward high masking regimes. In [8] they systematically evaluated alternative mask-rate schedules for continuous-loss MAR architectures, demonstrating that an exponential mask-rate distribution yields superior generative performance and enhances training efficiency within SD-VAE. Specifically, they implemented the exponential schedule with:

$$\gamma_s = 1 - \exp(-5s), \quad (3.18)$$

where s is sampled uniformly $s \sim \mathcal{U}[0, 1]$. This results in the density:

$$p(\gamma_s) = \frac{1}{5(1 - \gamma)} \quad (3.19)$$

for $\gamma_s \in [0, 1 - e^{-5}]$. This mask-rate schedule spends much more time on sampling high masking rates. We plot the density for the exponential mask-rate schedule in Fig. 3.3b with clamping s at different minimums $s \sim \mathcal{U}(s_{\min}, 1)$.



(a) Truncated normal distribution restricted to the interval $[r_{\min}, 1.0]$.

(b) Exponential distribution schedule.

Figure 3.3.: Comparison of different PDFs for mask-rate sampling schedules.

3.4. Generation Guidance

To enhance the structural fidelity and class alignment of generative models, we apply CFG [56, 57]. Under a standard conditional framework, the posterior distribution of a condition c given the data x is described via Bayes-rule as:

$$p(c | x) = \frac{p(x | c) \cdot p(c)}{p(x)}. \quad (3.20)$$

In practice, a single unified neural network is trained to simultaneously estimate the conditional velocity field $v_{\Theta}(x_t, t, c)$ and the unconditional counterpart $v_{\Theta}(x_t, t, \emptyset)$, where $\emptyset$ denotes a null conditioning. This capability is achieved by randomly dropping the conditioning signal c during the training phase according to a fixed probability (set to a baseline of 10% in our experiments). During inference, the guided velocity field $\hat{v}_{\Theta}$ is evaluated using the guidance scale parameter w with:

$$\hat{v}_{\Theta} = v_{\Theta}(x_t, t, \emptyset) + w \cdot (v_{\Theta}(x_t, t, c) - v_{\Theta}(x_t, t, \emptyset)). \quad (3.21)$$

Setting $w = 1.0$ recovers the standard conditional flow-matching trajectory, whereas $w = 0.0$ yields a purely unconditional generation path. Scaling $w > 1.0$ systematically trades off sample diversity in favor of strict adherence to the conditioning input.

Autoregressive Guidance

Extending this formulation to our localized, masked autoregressive setup requires adapting the guidance mechanism to operate on a per-token basis. In this paradigm, the conditioning signal c is replaced by the reconstructed global context vector z_i provided by the transformer backbone. Consequently, the unconditional null state $z_{\emptyset}$ is represented without class information. The guided velocity field for a target token index i is thus formulated with:

$$\hat{v}_{\Theta} = v_{\Theta}(x_{i,t_i}, t_i, z_{\emptyset}) + w \cdot (v_{\Theta}(x_{i,t_i}, t_i, z_i) - v_{\Theta}(x_{i,t_i}, t_i, z_{\emptyset})). \quad (3.22)$$

Progressive Autoregressive Guidance

A distinct advantage of per-token guidance is the ability to dynamically adapt the scale factor w based on the generation progress. Because earlier tokens establish the coarse, global layout, a static guidance scale may over-constrain or under-guide specific phases of the autoregressive process. To address this, we follow the progressive scheduling strategy introduced by MAR and formalize the guidance scale as a function of the current autoregressive step k . Specifically, the dependent

scale w_k is modulated by the ratio of visible tokens to the total sequence length N . We can define the scale w_k with:

$$w_k = 1 + (w_{\text{linear}} - 1) \frac{M_k}{N}, \quad (3.23)$$

where M_k is the number of unmasked context tokens available at step k , N is the total sequence length and w_{linear} represents the maximum target guidance scale. Incorporating this dynamic weight into our per-token formulation yields the final progressive guidance objective:

$$\hat{v}_{\Theta} = v_{\Theta}(x_{i,t_i}, t_i, z_{\emptyset}) + w_k \cdot (v_{\Theta}(x_{i,t_i}, t_i, z_i) - v_{\Theta}(x_{i,t_i}, t_i, z_{\emptyset})). \quad (3.24)$$

By parameterizing w_k to monotonically increase, the schedule enforces a loose, high-diversity constraint during initial structural placement when M_k is small. This is followed by aggressive conditioning adherence in the final stages of token generation as $M_k \rightarrow N$, maximizing local perceptual fidelity.

3.5. Analysis of Different Representation Spaces

In this section we define distinct target representations in which we want to explore our masked flow-matching model. We first define our choices for representation spaces and in the next subsection we want to analyze and compare them.

3.5.1. Definition of Representation Spaces

Following the setup in MAR [7], we keep the sequence length constant at $N = 256$ tokens across all spaces. In our work we consider the following three types of diverse representations.

Stable Diffusion VAE (SD-VAE). As a baseline, we employ the SD-VAE introduced by [1]. This architecture has long served as the standard tokenizer for continuous image generation models. The primary objective of this tokenizer is to significantly compress spatial pixel data. For instance, an SD-VAE variant with a stride of 16 compresses a 256×256 pixel image into a latent space of dimensions $16 \times 16 \times 16 = 256 \times 16$, yielding a substantial spatial compression factor. For our experiments, we select the KL-regularized variant with a stride of 16, ensuring direct compatibility and fair comparison with the tokenizer utilized in MAR.

Representation Autoencoders (RAE). We further investigate pre-trained and frozen vision foundation models, trained via self-supervised learning, to establish a semantically rich representation space. This class of tokenizers is referred to as RAEs. Given an input image $x \in \mathbb{R}^{3 \times H \times W}$, encoding it with an RAE encoder yields

$N = H \cdot W / p_E^2$ tokens, where p_E denotes the patch size of the encoder. To evaluate the impact of different training objectives, we select two distinct configurations: DINOv2-B ($d = 768$) [3], which is trained via self-supervised and self-distillation, and MAE-B [4], which relies on a masked pixel-reconstruction objective. To reconstruct pixels, we utilize the pre-trained decoders proposed by [5]. These decoders are optimized using a joint objective with l_1 loss, an LPIPS perceptual loss, and an adversarial (GAN) loss. For implementation details regarding the decoders, see Appendix A.1.

Raw Pixel Space. Inspired by recent advancements showing that end-to-end training in pixel space without intermediate tokenization yields remarkable performance in diffusion transformers [6], we also explore the raw pixel space within our masked flow-matching model. To achieve this, the input image $x \in \mathbb{R}^{3 \times H \times W}$ is patchified into a sequence of tokens. This results in a sequence length of $N = H \cdot W / p^2$ and a flattened patch embedding dimension of $d = 3 \cdot p^2$, where p denotes the spatial patch size. Following the generative process, the predicted continuous tokens are mapped back to the original image space via a straightforward unpatchify operation to obtain back the final output shape $x \in \mathbb{R}^{3 \times H \times W}$.

3.5.2. Metrics for Representation Spaces

In this section we want to analyze and compare our representations. We start to define metrics for good representations. Then we will compare different representations with this metrics and take this as a assumption for our experiments.

In image generation we want to learn on the one hand how pixels form textures and shapes. On the other hand, we want to learn the meaning of the condition, so in class-to-image generation this is what the class defines. So for the design of a good latent space the following two properties should be take into account:

- **Perceptual Fidelity:** Ensures that the images are sharp and have good fine details
- **Learnability:** The latent are meaningful and the feature are disentangled

As the perceptual fidelity is responsible for the final quality of the generated image this is an important metric for a tokenizer. A second important metric is the learnability of that latent space, what means if our latent space has a good semantic structure we can easier learn the transport from noise to data and our training will be more efficient. In the following two paragraphs we will discuss this two metrics for different tokenizers.

Perceptual Fidelity. We begin by analyzing the reconstruction fidelity of our representation spaces as reconstruction quality fundamentally limits the generative performance. To quantify perceptual fidelity, we employ the LPIPS metric with a VGG feature extractor [50]. As we can see in Tab. 3.1, the highest reconstruction

quality is achieved by the pixel-reconstruction-based MAE, closely followed by the SD-VAE baseline. Conversely, both DINOv2 variants exhibit notably higher LPIPS distances. We complement this with the Reconstruction Fréchet Inception Distance (rFID) metric proposed by [5], as detailed in Tab. 3.1. We can observe that the two metrics rFID and LPIPS differ for DINOv2 encoders. This observation aligns with recent findings by [34] and it can indicate that DINOv2 is suitable for generation tasks in general, as the rFID is good. However, as the LPIPS score is notably worse it can lack in high-fidelity reconstructions.

Encoder	Dim.	LPIPS ↓	rFID ↓
SD-VAE (KL-16)	16	0.14	0.62
MAE-B	768	0.11	0.16
DINOv2-B	768	0.255	0.49
DINOv2-L	1024	0.266	0.52

Table 3.1.: Metrics for reconstruction quality of selected tokenizers.

In Fig. 3.4 we visualize the reconstructed image of five classes of ImageNet after we applied our autoencoders on them. We can clearly see that both scales of DINOv2 lacks behind SD-VAE and MAE-B, while MAE-B shows the overall best reconstructions.

Analysis of Semantic Structure. The second property to quantify a good representation space for image generation is its inherent learnability. While this attribute can be measured directly by tracking the Generation Fréchet Inception Distance (gFID) across training iterations, we want to provide a preliminary qualitative analysis. A common property for estimating learnability is the semantic structure of the underlying representation space. A well-structured space yields clearly separated and distinct target trajectories, which significantly decreases the optimization effort on the generative model. Consequently, if a representation space exhibits a robust semantic layout, the conditioning phase becomes easier for the generative model to learn, leading to faster training convergence.

To evaluate these semantic structures, we follow the established dimensionality reduction methodologies of [58, 59] and employ the t-Distributed Stochastic Neighbor Embedding (t-SNE) algorithm to project high-dimensional representations into an interpretable two-dimensional space. Specifically, we randomly sample 20 distinct classes from the ImageNet dataset, selecting 200 images per class. Each image is encoded via the respective latent encoder, followed by an average-pooling operation over all sequence tokens. Therefore the resulting global representations are subsequently reduced to two dimensions via t-SNE and visualized in Fig. 3.5.

For the pixel space, we patchify each 256×256 pixel image into non-overlapping patches with a resolution of 16 pixels, resulting in a reshaped tensor of shape

3.5. Analysis of Different Representation Spaces

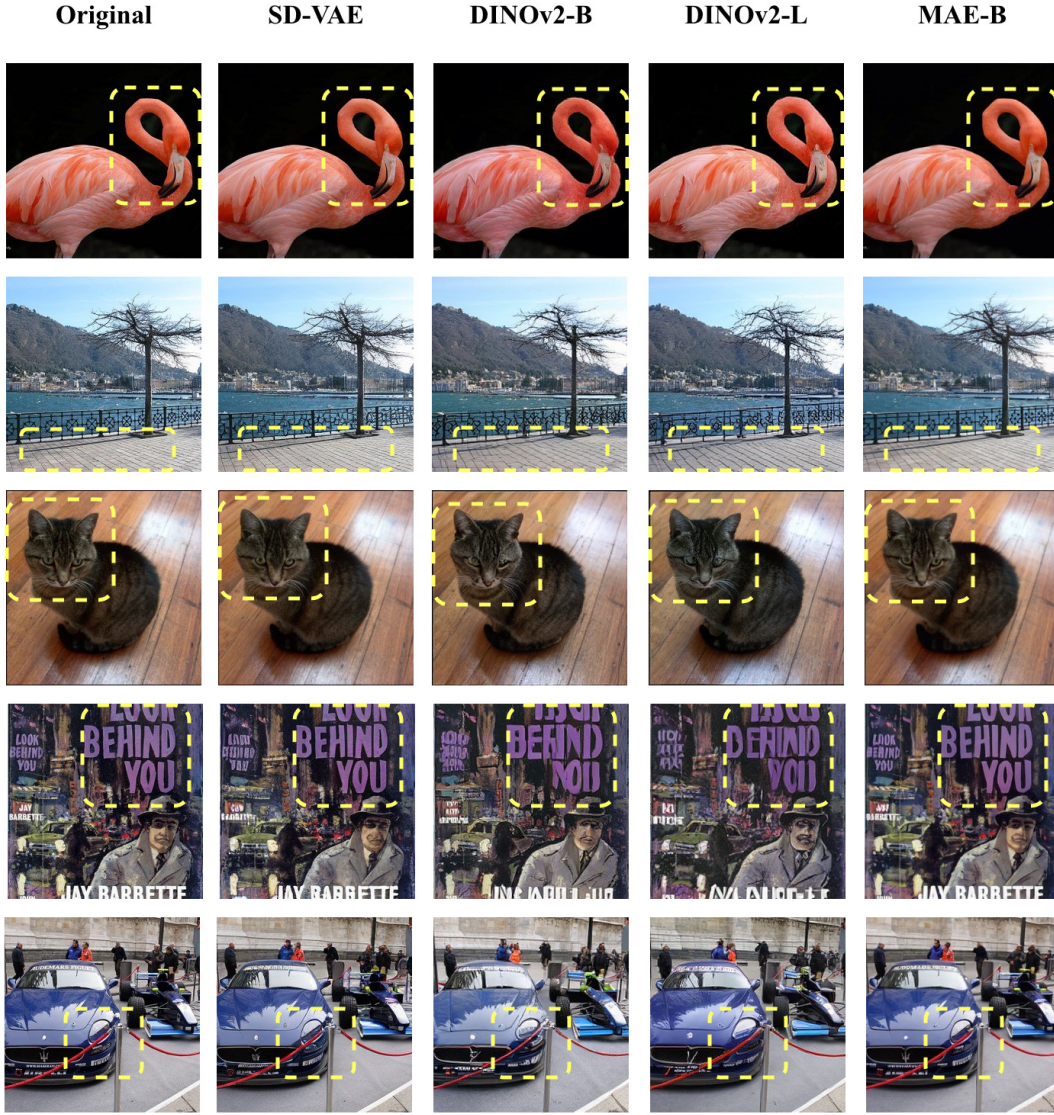


Figure 3.4.: Comparison of reconstructions using the autoencoders (SD-VAE, DINOv2-B, DINOv2-L, MAE-B) for four representative ImageNet examples.

$[B, 256, 16 \times 16 \times 3]$. Under this setup, executing an average-pooling operation across all spatial tokens nearly collapses the sequence into global image mean color statistics. As a result, the pixel manifold captures no semantic relationships, exhibiting a completely unclustered distribution in Fig. 3.5a, which aligns with our expectations.

Similarly, because the SD-VAE is optimized via a pixel-level reconstruction loss regularized by a KL divergence penalty, its latent tokens are bounded to approximate a standard normal distribution $\mathcal{N}(0, I)$ across their 16-dimensional channels.

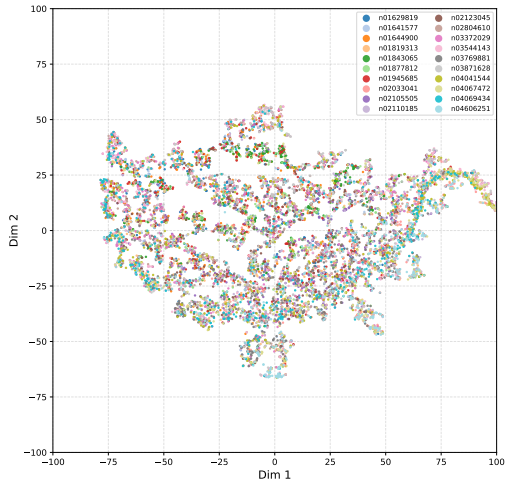
This strong regularization forces the manifold into a highly compact, centralized configuration without distinct class clusters, as visualized in Fig. 3.5b.

In contrast, utilizing vision-foundation models such as MAE or DINOv2, reveals highly isolated class clusters (see Fig. 3.5c and 3.5d). Despite their fundamentally different self-supervised training paradigms, both foundational encoders successfully construct a highly organized, semantically coherent representation space.

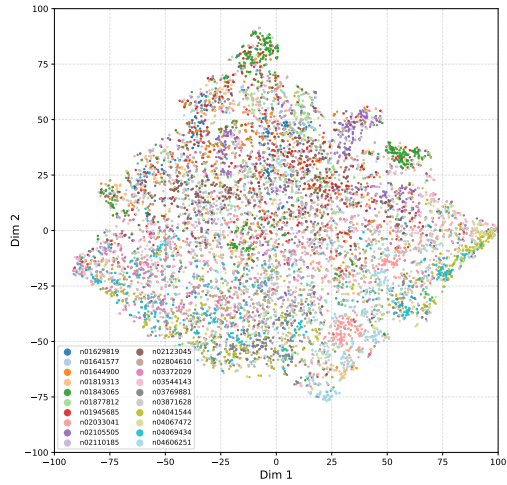
3.5.3. Masking in Different Spaces

In Fig. 3.6 we visualize each representation space alongside its corresponding decoded spatial image after a masking schedule has been applied directly within that space. This visualization allows us to systematically understand how spatial masking effects the preserved information across different semantic configurations. We see that in SD-VAE and DINOv2 a single token effects also neighboring tokens while MAE does not.

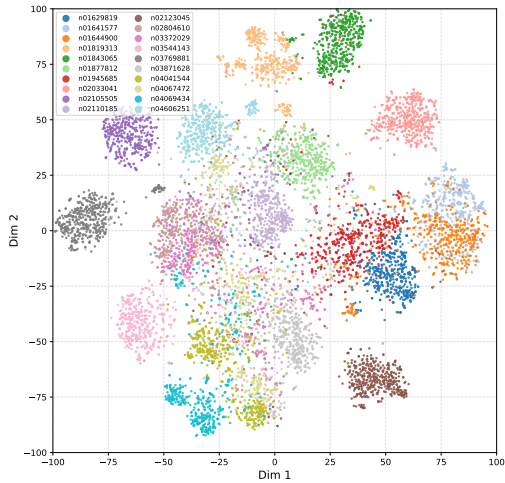
3.5. Analysis of Different Representation Spaces



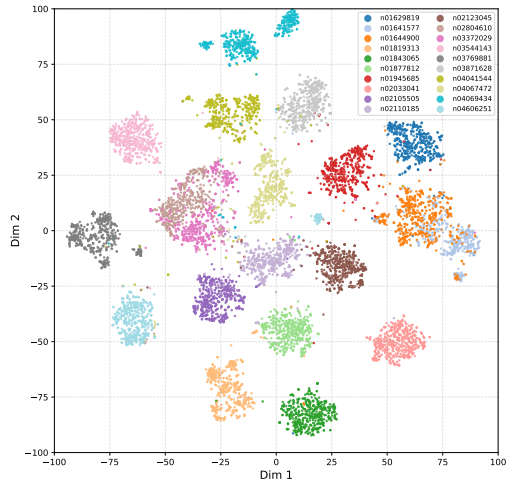
(a) Pixel Space (dim = 768), where spatial average-pooling collapses tokens into color/statistical means, showing no categorical cluster grouping.



(b) SD-VAE (KL-16) (dim = 16), illustrating a dense, isotropic distribution regularized toward $\mathcal{N}(0, I)$ to prioritize perceptual instead of semantic bounds.



(c) MAE-B (dim = 768), highlighting distinct cluster configurations as a result of a masked reconstruction objective.



(d) DINOv2-B (dim = 768), showing highly separated categorical clusters driven by its self-supervised and self-distillation objective.

Figure 3.5.: Qualitative evaluation of semantic structure across alternative image representation spaces using two-dimensional t-SNE projections of 20 randomly selected ImageNet classes with 200 samples each.

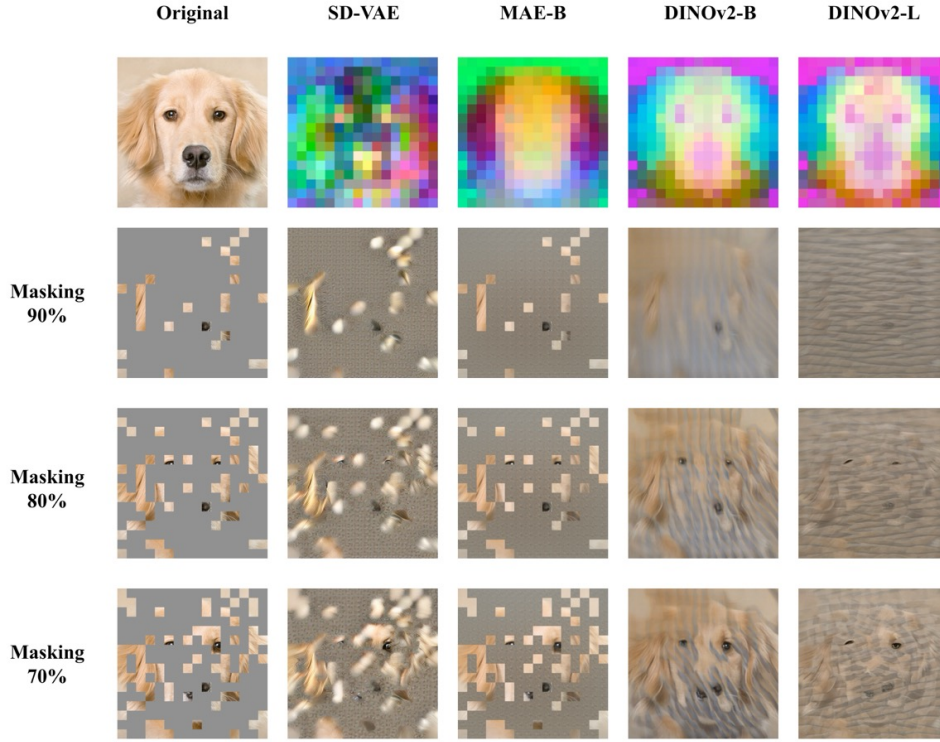


Figure 3.6.: Visual comparison of different representation spaces under spatial masking schedules. We present our encoded representation spaces alongside its corresponding decoded spatial reconstruction after token masking has been applied directly within that respective space.

3.5.4. Assumptions for Experiments

Through our analysis of alternative representation spaces, we observe that a semantically meaningful latent manifold can also possess the structural properties necessary to achieve high-fidelity reconstructions. Furthermore, a highly organized semantic structure is expected to be beneficial for the generative model, thereby increasing training efficiency. For our continuous masked flow-matching architecture, a primary technical challenge lies in effectively modeling these high-dimensional feature spaces. We hypothesize that the raw pixel space represents the most challenging domain to optimize, as this manifold is completely unstructured while simultaneously retaining a high dimensionality. Despite these optimization hurdles, we include the pixel space in our experiments, as bypassing an tokenization stage yields to a end-to-end image generation pipeline that avoids the architectural biases of decoupled decoders.

4. Model Implementation

In this chapter we discuss the specific implementation of our defined masked flow-matching model which we will use for our experiments. We first define the transformer backbone which builds the autoregressive part and provides the global context. Then we will define the per-token denoiser which implements the conditional distribution $p(x_i|z_i)$. Finally we will present the specific model parameters and training details.

4.1. Implementation of the Transformer Backbone

Defining Masked Autoregression. We follow MaskGIT [42] and implement the autoregression by using a transformer [18] with bidirectional attention, which allows all tokens to see each other. Combined with positional embedding the decoding can be learned with bidirectional attention as then all masked tokens see all unmasked tokens. Therefore the autoregression is defined over the masked and not like common autoregression by causal attention. As visualized in Fig. 4.1, the "next-token-prediction" here follows a random order defined by the mask. At inference we can generate multiple tokens together and therefore reduce the generation steps.

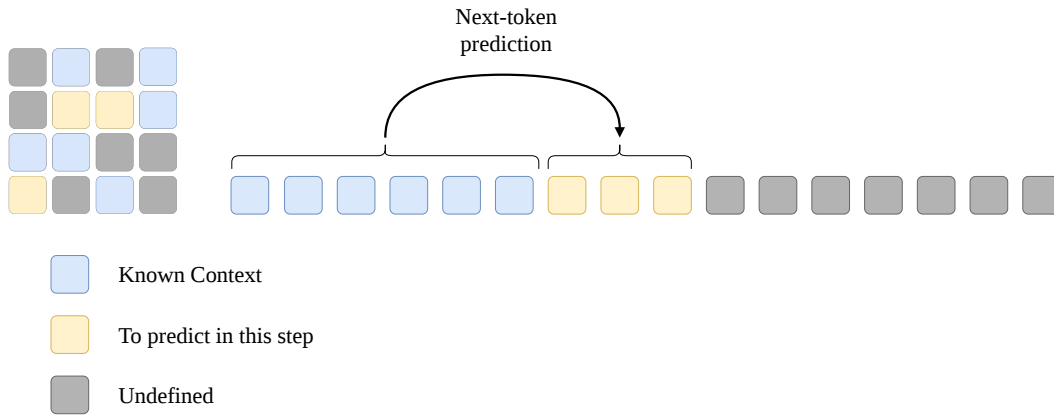


Figure 4.1.: Structural schematic of the masked autoregressive generation process.

Transformer Architectures. In MAR the architecture is defined with a encoder-decoder structure as visualized in Fig. 4.2a. The idea from MAR is to first extract meaningful features from the compressed SD-VAE latents and then train a decoder by

adding learnable mask-tokens to the token sequence. This idea is aligned with MAE [4], where the encoder is needed to produce a useful feature-space in which a decoder can learn to reconstruct the masked tokens. As we want to explore among other representation spaces its an open question if we need a trainable feature encoder or if we can directly train a decoder on the masked features of the frozen latent encoder as visualized in Fig. 4.2b. In the decoder-only version of the transformer backbone we direly concatenate the class tokens to the decoding token sequence.

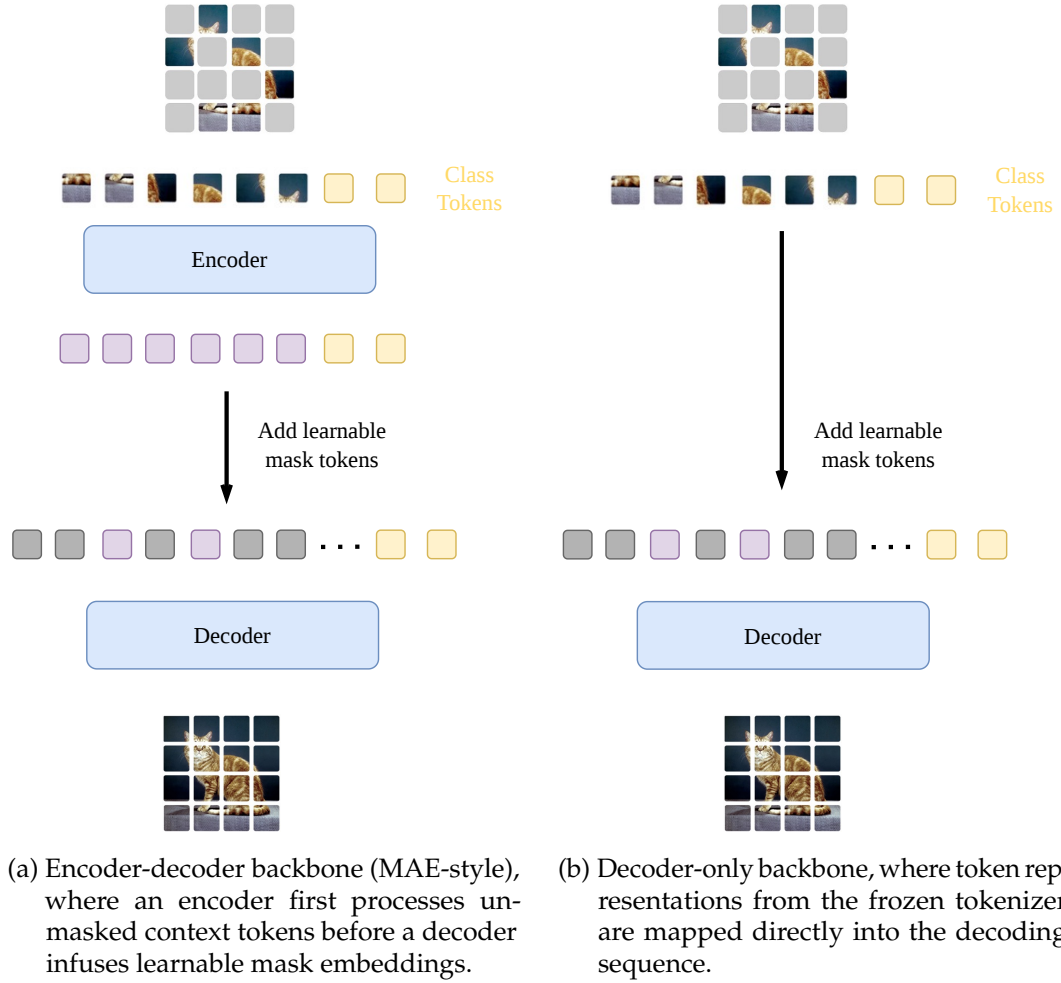


Figure 4.2.: Comparison of different transformer backbone architectures for our masked autoregressive flow pipeline.

Transformer Implementation. As we use a common transformer architecture as define in [16] we can benefit from architectural advances known from language models and other related tasks. We used SwiGLU [60], RMSNorm [61] and RoPE [62]. Following MAR we also in-cooperate the class tokens as in-context tokens to the transformer sequence. We want to note that in our pipeline the denoiser does not get

directly the class information, it gets global context as well as the class-information via the condition z of the transformer backbone.

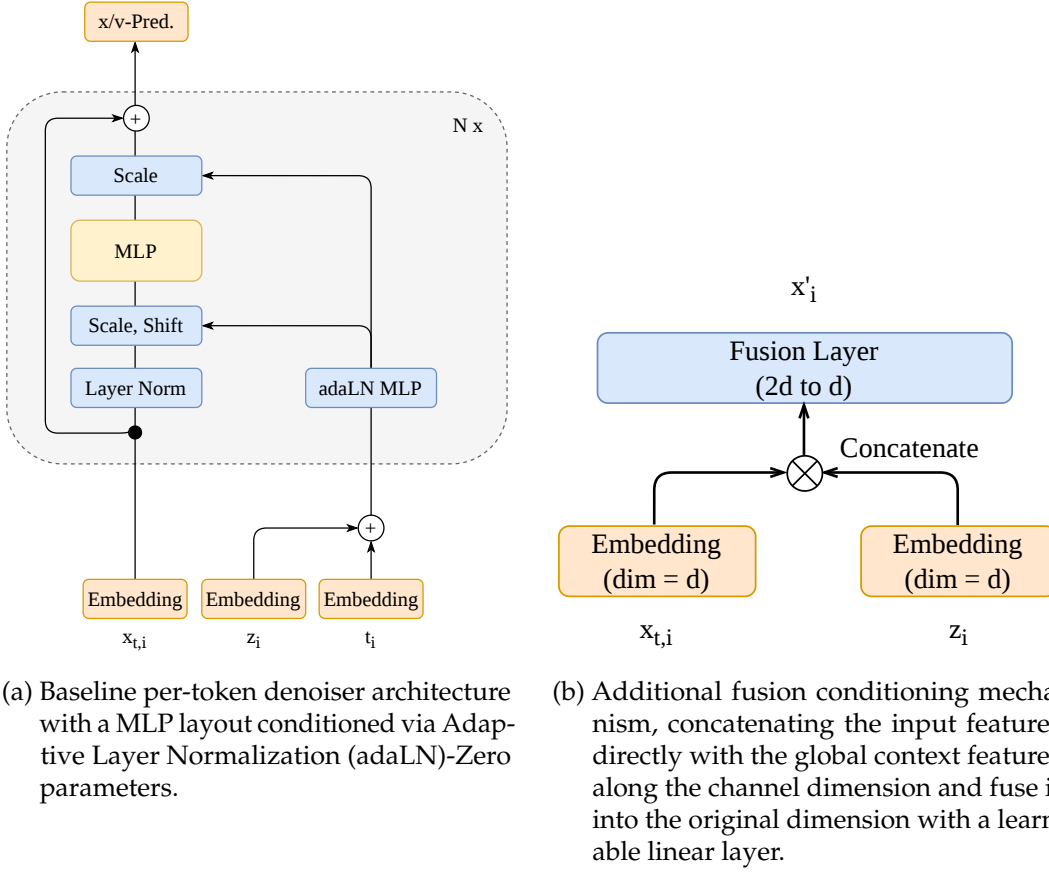


Figure 4.3.: Architecture for the localized per-token denoiser network which takes the noisy data token $x_{t,i}$ and the global context token z_i as input.

4.2. Implementation of the Denoiser

Conditioning Mechanism. The localized per-token denoiser relies on the adaLN mechanism introduced by [19], which builds upon [63]. This conditioning strategy scales and shifts normalized intermediate features using linear parameters optimized by a neural network:

$$x' = \gamma_{\Theta} \odot x + \beta_{\Theta}, \quad (4.1)$$

where $\odot$ denotes element-wise multiplication, and the scale and shift vectors are dynamically predicted via a Multi Layer Perception (MLP) such that $[\gamma_{\Theta}, \beta_{\Theta}] =$

MLP(c). In our continuous masked flow-matching pipeline, the input conditioning signal c is the combination of the global context token z_i and the timestep t_i . More precisely, we employ the adaLN-Zero variant, where the final layer of the conditioning MLP and the residual gating parameters are initialized to zero, forcing the blocks to initially act as identity mappings to stabilize early training steps.

Baseline Architecture. In Fig. 4.3a, we outline our core per-token denoiser architecture, adapted from MAR. Because this component models the denoising trajectory of an individual token i independently, it is parameterized entirely using a dense MLP structure without computationally heavy attention layers. The architecture takes the corresponding noised token state $x_{t,i}$ as input and use adaLN-Zero as conditioning. To facilitate robust gradient propagation, we incorporate block-wise residual connections [64].

Additional Denoiser Conditioning. To expand beyond pure normalization-based modulation, we evaluate an additional conditional strategy for the global context vector z_i . As illustrated in Fig. 4.3b, instead of restricting the context to a layer-norm modulation, we explicitly concatenate the noisy token input $x_{t,i}$ and the global context embedding z_i along their channel dimension. Then, a learnable fusion layer is applied to map the expanded channel dimension d' back into the original network hidden dimension d . We systematically explore the empirical benefit of this architectural modification in our experiments.

4.3. Model Parameters and Training Details

In Tab. A.2 we present the used model architectures as well as training and sampling parameters for all three representation spaces. We note that we will explore different configurations during our experiments in each space.

5. Experiments on Different Spaces

In this chapter, we evaluate our defined masked flow matching model across representation spaces. We conduct conditional class-to-image generation using the ImageNet-1k dataset [65] at a resolution of 256×256 . For quantitative evaluation, we utilize the torch-fidelity codebase [66] and strictly follow the evaluation protocols for generative models established by [67]. To compute the FID and IS metrics, we leverage the precomputed statistics from JiT [6].

All evaluations are performed using 50k generated samples. For faster ablation studies, we only use 10k samples, which we then explicitly denote. The primary objectives of our experiments are to explore the selected representation spaces, their parameters, and effective architectures within the context of masked flow matching. Additionally, we aim to analyze the generation process and investigate the effectiveness of CFG guidance for each space.

5.1. Baseline

We begin our experiments by establishing a baseline using the well-known SD-VAE tokenizer from [1]. We therefore select the exact same SD-VAE variant from MAR, specifically the KL-16 variant for continuous tokens. Here, the factor of 16 denotes the tokenizer stride, which yields 16×16 tokens with a channel dimension of 16.

We follow the training and implementation configurations detailed in Table A.2. We begin with uniform time sampling, $t \sim \mathcal{U}([0, 1])$, which is used in MAR. Fig. 5.1 provides the FID and IS metrics over of training iterations.

Next, we investigate whether a logit-normal timestep distribution benefits our masked flow matching model, as its common practice for joint flow matching transformers [9]. The logit-normal distribution concentrates sampling density on intermediate noise levels while allocating less weight to the boundaries ($t = 0$ and $t = 1$). As demonstrated in Fig. 5.1, adopting the logit-normal distribution consistently improves generation quality throughout the entire training process.

Optimal CFG Scale

Next, we explore the guidance strategy for our baseline by evaluating different CFG scales w as defined in Eqn. 3.22. For each scale w , we generate 10k samples and

Chapter 5. Experiments on Different Spaces

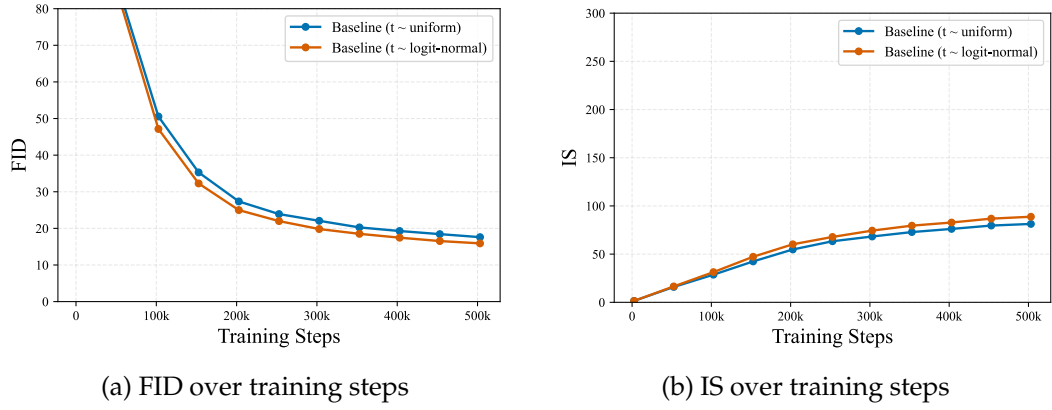


Figure 5.1.: Baseline evaluation of convergence within the SD-VAE latent space utilizing a uniform and logit-normal timestep distribution.

compute the FID. Fig. 5.2a presents the performance across different scales for both the linear and progressive strategies. Progressive means that the CFG scale is first applied during the later autoregressive steps as defined in Eqn. 3.24.

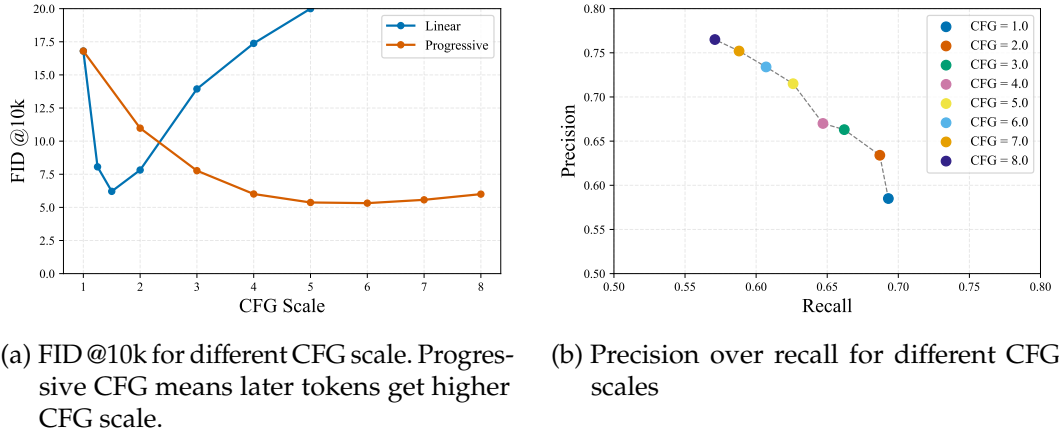


Figure 5.2.: Comparing several CFG scales to find the optimal generation quality within the SD-VAE latent space.

Our ablation indicate that the SD-VAE latent space benefits significantly from the progressive CFG strategy, with a scale of $w = 6.0$ achieving the best performance. Additionally, we evaluate the precision-recall curve across different CFG scales (see Figure 5.2b). As is typical for CFG, we observe a trade-off where sample diversity is sacrificed to achieve higher fidelity.

Generated Examples with SD-VAE

We present visual samples generated utilizing the SD-VAE latent space within our baseline model configuration. Based on our previous observation, we employ a progressive guidance schedule with a maximum CFG scale of $w = 6.0$, as it yields the optimal trade-off between perceptual fidelity and structural diversity. A representative selection of these generated samples is illustrated in Fig. 5.3. This configuration achieves an FID score of 3.18 and an IS of 264.74. Additional qualitative results are provided in Appendix B.1.

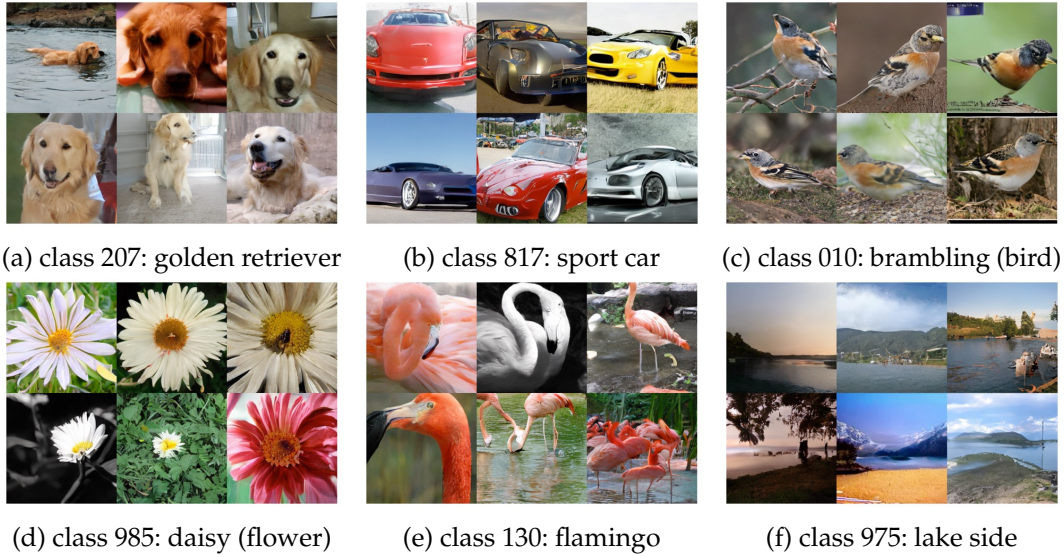


Figure 5.3.: Generated examples using the SD-VAE latent space. We trained for 200 epochs and used the best performing CFG with a progressive scale of $w = 6.0$. This generated images results in FID=3.18 and an IS= 264.74.

5.2. RAE Latent Space

Our next representations are the high-dimensional and semantically meaningful RAEs. We begin this exploration by comparing different parameters and architectures, systematically analyzing which components improve generation quality and efficiency. Throughout this analysis, we refer to the results in Tab. 5.1. As a starting point, we adopt the same hyperparameters and architecture used for our baseline. Since RAE latents inherently exhibit strong semantic features, we begin with just two encoder layers for the transformer backbone of our masked generative model. We later compare this setup against a decoder-only architecture. To ensure a fair comparison across all representation spaces, we maintain a consistent number of trainable parameters across all spaces.

Prediction Type. In this comparison, we aim to explore two specific questions: (i) Does the data manifold of semantically meaningful RAEs (such as DINOv2) also lie on a lower-dimensional manifold compared to Gaussian noise. (ii) Can the conditional distribution of our masked flow matching model effectively utilize the lower-dimensional manifold. For x-prediction, we use Eqn. 3.8 and Eqn. 3.9 to reparameterize the network output into a velocity loss. In Tab. 5.1, we compare both configurations. Contrary to our initial expectations, we observe that x-prediction does not improve performance under equal model parameters. Instead, the generation quality is slightly worse than with v-prediction.

Timestep Schedule. In our next set of experiments, we compare timestep distributions. First, we select the logit-normal distribution as defined in Eqn. 3.15 and Eqn. 3.16. Second, we evaluate with a distribution shift defined in Eqn. 3.17 with a scale parameter of $\alpha = 6.93$ (optimal in shift for RAEs [5]). The results for both configurations are presented in Tab. 5.1. As we can see, incorporating the logit-normal timestep distribution alongside the distribution shift significantly improves the overall generation quality.

Autoregressive Reconstruction Architecture. Since RAE features are inherently semantically meaningful, we investigate whether trainable encoder layers remain necessary to extract features for the transformer decoder. As shown in Tab. 5.1, a shallow trainable encoder proves beneficial. We hypothesize that this improvement is primarily driven by the injection of the class tokens, as the two encoder layers effectively integrate them into the token sequence. Finally, we note that our approach still saves a substantial portion of model parameters with RAEs. In contrast, the baseline requires an equal number of layers (10 layers) for both the encoder and the decoder.

Denoiser Capacity and Conditioning. Because RAEs possess a significantly larger embedding dimension (e.g., DINOv2-B with $\text{dim} = 768$) compared to SD-VAE ($\text{dim} = 16$), we explore whether the MLP denoiser has sufficient capacity and whether the adaLN-conditioning of the global latent z from the transformer is powerful enough. To address this, we introduce a fusion of z and x at the input of the

	Parameter	Steps	FID ↓	IS ↑
VAE latent space				
logit-normal (flow loss) (w/o CFG)	179.6M	503k	15.91	88.78
logit-normal (flow loss) (CFG=6.0)	179.6M	503k	3.18	264.74
DINOv2 latent space				
Prediction type				
v-prediction	125.6M	503k	9.54	186.85
x-prediction	125.6M	503k	10.70	158.70
Timestep schedule				
uniform time sampling	125.6M	503k	9.54	186.85
logit-normal + shift ($\alpha = 6.93$)	125.6M	503k	7.59	211.37
Reconstruction Architecture				
decoder only	124.7M	503k	8.55	204.48
encoder (2 layer) + Decoder	125.6M	503k	7.59	211.37
Denoiser and Conditioning				
adaLN (dim=1024, ratio 1)	124.6M	503k	7.59	211.37
adaLN (dim=1024, ratio 4) + z-fusion	165.4M	503k	6.30	243.91
adaLN (dim=1536, ratio 4) + z-fusion	255.1M	503k	5.33	248.00

Table 5.1.: Step-by-step comparison of distinct model parameters and architecture within the **DINOv2** latent space. We trained all models for 200 epochs (503k training iterations) and evaluate over 50k samples.

denoiser while simultaneously retaining z within the adaLN blocks. Furthermore, we give the denoiser a higher per-block dimension. This modification provides the block with a higher non-linear capacity, enabling it to capture more complex feature patterns. A comparison of these configurations is presented again in Tab. 5.1. In a third variant, we increase the denoiser dimension from 1024 to 1536 while keeping all other parameters constant. Our results indicate that scaling up the denoiser capacity is crucial when dealing with RAEs, which we attribute to the challenges of processing high-dimensional features. For the remainder of our comparative analysis, however, we maintain the 1024-dimensional configuration to keep the total number of model parameters comparable to our baseline.

Different RAE Encoder Objectives

Next, we compare fundamentally different RAE encoders. While DINOv2 is trained using a self-supervised, self-distillation objective, MAE utilizes a masking and reconstruction objective. Prior work by [5] observed that the MAE latent space is not well-suited for joint flow matching generative models. However, because our model

employs a masked, per-token flow matching objective, we investigate whether MAE features might prove effective under this formulation.

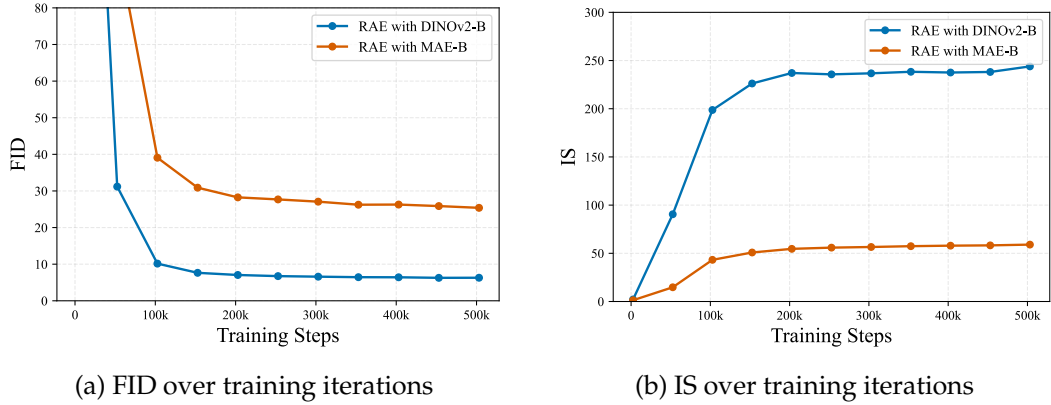


Figure 5.4.: Comparing two fundamental different features encoders within the RAE latent space. We compare the generation quality of DINOv2 and MAE latent space.

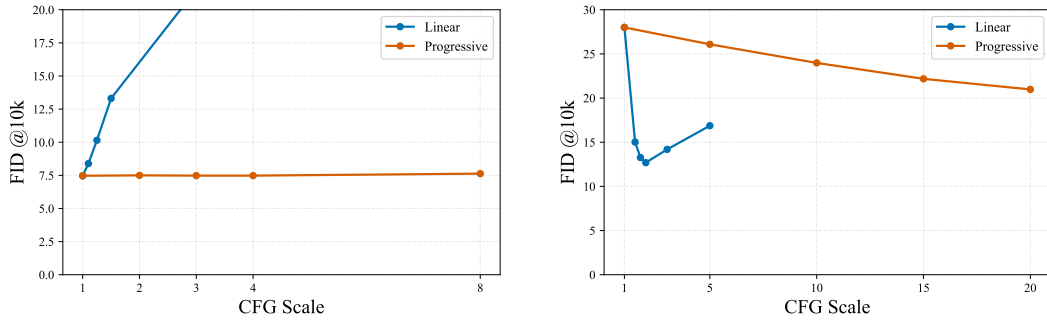
For this experiment, we select the same hyperparameters and architecture as in the final experiment for DINOv2 in Tab. 5.1. In Fig. 5.4, we plot the FID and IS metrics over training iterations. The results clearly demonstrate that DINOv2 outperforms MAE features in masked autoregressive models too. We further observe that the convergence speed remains comparable between both spaces, which aligns with expectations given that both latent spaces possess a strong semantic structure.

CFG Guidance

In Fig. 5.5a, we report the CFG behavior for the DINOv2 latent space. We sample 10k images and calculate the FID for each CFG scale w using both the linear guidance from Eqn. 3.22 and the progressive guidance from Eqn. 3.24. Surprisingly, applying a CFG scale does not improve the FID for DINOv2 at all. Even the progressive approach, which applies stronger scales to later tokens, fails to yield any FID improvements.

We conducted the same set of experiments for the MAE latent space, with the results illustrated in Fig. 5.5b. In contrast to DINOv2, these experiments demonstrate that MAE latents heavily depend on strong guidance. Specifically, CFG improves generation quality from an FID of 28.00 down to 12.69 at a scale of $w = 2.0$. Furthermore, we observe that even the initial tokens require strong guidance, as the progressive variant does not show significant improvement even at high CFG scales ($w > 10.0$). From these two experiments, we can assume that while guidance can be beneficial for RAEs, its effectiveness is closely tied to the training objective of the underlying feature encoder.

5.2. RAE Latent Space



(a) FID @10k for different CFG scale for DINOv2 latent space. (b) FID @10k for different CFG scale for MAE latent space.

Figure 5.5.: Comparing the CFG scale within the DINOv2 and MAE latent space.

Generated Examples in RAE (with DINOv2)

In Fig. 5.6, we present examples generated using the DINOv2 latent space. Since our previous experiments indicated that CFG guidance does not improve the FID, we maintain a scale of $w = 1.0$. These samples achieve an FID = 5.33 and an IS = 248.00. We present more samples in Appendix B.2.



(a) class 207: golden retriever



(b) class 817: sport car



(c) class 010: brambling (bird)



(d) class 985: daisy (flower)



(e) class 130: flamingo



(f) class 975: lake side

Figure 5.6.: Generated examples using the **DINOv2** latent space. We trained for 200 epochs and didn't used CFG-scale as we found that we worsen the FID. This generated images results in FID=5.33 and an IS=248.00

5.3. Pixel Space

In pixel space we start with the logit-normal timestep distribution and a transformer backbone with an equal number of layers for both the feature encoder and decoder. Furthermore, we begin with the exponential mask-rate distribution defined in Eqn. 3.18. Similar to RAEs, we systematically explore different parameters and model architectures optimized for the pixel space. We provide the results in Tab. 5.2. We note that these experiments are compared without CFG guidance. We ablate and explore the optimal CFG scale in a subsequent analysis.

Prediction Type. In Tab. 5.2 we provide a comparison of both prediction types. Contrary to the observations reported in JiT, we find that v-prediction does not fail within masked generative models. However, x-prediction still proves beneficial. We note that the overall poor generation quality at this stage arises because raw pixels heavily depend on strong CFG guidance. Consequently, for all subsequent pixel-space experiments, we utilize x-prediction.

Mask-rate Schedule. Next, we investigate the choice of mask-rate distribution. While our baseline configurations rely on aggressive exponential masking, we find that a less aggressive masking schedule is beneficial for pixel space generation (see Tab. 5.2), unlike with RAEs and SD-VAE. We hypothesize that because pixel tokens are inherently localized, the model requires a larger amount of context to capture spatial dependencies effectively. To address this, we employ the truncated normal distribution. We note, however, that the optimal masking strategy for raw pixel representations warrants deeper future exploration.

Bottleneck Embedding. Related to JiT we explored with a bottleneck embedding layer. More specifically we replace the linear projection embedding of the denoiser’s input into a pair of linear bottleneck layers. The first layer reduces the dimension to d' and the second layer expands it to the hidden size of the denoiser. This serves as a low-rank reparametrization and should encourage the learning of the low-dimensional clean data in the high-dimensional noise space. We explored that a bottleneck dimension $d' = 128$ is useful in pixel space. We keep this bottleneck dimension for the next runs.

Denoiser Capacity and Conditioning. Next, we want evaluate whether the conditioning mechanism and capacity of the baseline MLP denoiser are optimally designed. Mirroring our approach with RAEs, we increase the per-block hidden dimension, providing the denoiser with additional capacity to learn complex, high-dimensional data patterns. Simultaneously, we introduce an input fusion as defined in Fig. 4.3b. Similar to our findings with RAEs, these experiment demonstrate that both, the capacity and the conditioning of the per-token denoiser are critical for high-dimensional raw pixels (see Tab. 5.2).

	Parameter	Steps	FID ↓	IS ↑
SD-VAE latent space				
logit-normal (flow loss) (w/o CFG)	179.6M	503k	15.91	88.78
logit-normal (flow loss) (CFG=6.0)	179.6M	503k	3.18	264.74
Pixel Space				
Prediction type				
v-prediction	182.3M	503k	104.69	15.82
x-prediction	182.3M	503k	89.64	21.10
Mask-rate schedule				
exponential	182.3M	503k	89.64	21.10
trunc-normal	182.3M	503k	85.05	23.36
Bottleneck layer				
no bottleneck	182.3M	503k	85.05	23.36
bottleneck $d'=128$	182.3M	503k	71.04	24.62
Denoiser and Conditioning				
adaLN (dim=1024, ratio 1)	182.3M	503k	71.04	24.62
adaLN (dim=1024, ratio 4) + z-fusion	221.6M	503k	42.39	41.04

Table 5.2.: Step-by-step comparison of distinct model parameters and architecture within the **pixel space**. We trained all models for 200 epochs (503k training iterations) and evaluate over 50k samples.

CFG Guidance

Next, we aim to determine the optimal CFG scale to guiding the generation within pixel space. Consistent with our evaluations in the other target spaces, we compute the FID over 10k generated samples across a range of CFG scales. As provided in Fig. 5.7, generation quality improves significantly when employing a linear CFG strategy. While we further experimented with a progressive guidance schedule, our findings indicates that a linear guidance schedule is required. Notably, we observe that CFG is absolutely essential when operating directly in pixel space. Based on these results, we fix the CFG scale at its optimal value of $w = 2.0$ for all subsequent experiments in pixel space.

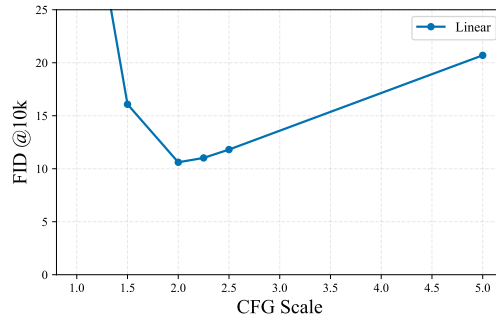


Figure 5.7.: FID@10k for different CFG scales in pixel space.

Generated Examples in Pixel Space

Finally, we present visual examples generated using the pixel space in Fig. 5.8. For these samples, we apply the optimal linear schedule CFG scale of $w = 2.0$. This configuration achieves an FID=8.70 and an IS=216.76. Visually, we observe that generation in the pixel space appear slightly more blurry and lack of fine details (we will come back to this point in the discussion chapter 6).

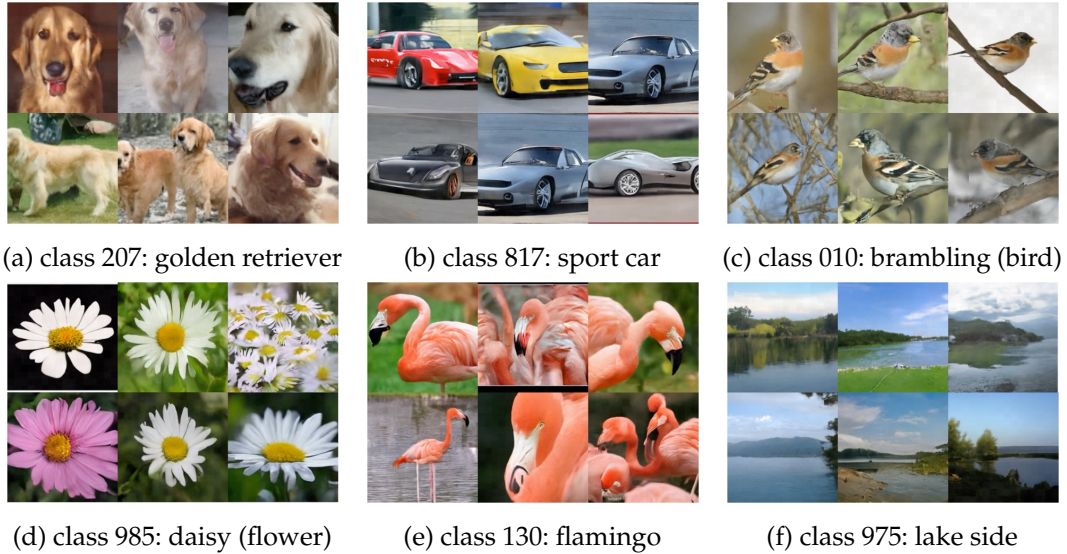


Figure 5.8.: Generated examples using the **pixel space**. We trained for 200 epochs and used the best performing CFG scale of $w = 2.0$ with a linear schedule. This generated images results in FID=8.70 and an IS=216.76

5.4. Comparison of Different Representation Spaces

In this section, we evaluate and compare the representation spaces under consideration. We first analyze their training efficiency, followed by an assessment of their final generation quality. To provide a more granular quantification of generation performance, we also introduce precision and recall. Furthermore, we investigate the impact of masking schedules during training and visualize the latent features of the transformer backbone. Finally, we examine the influence of various sampling parameters and visualize the intermediate stages of the autoregressive generative process.

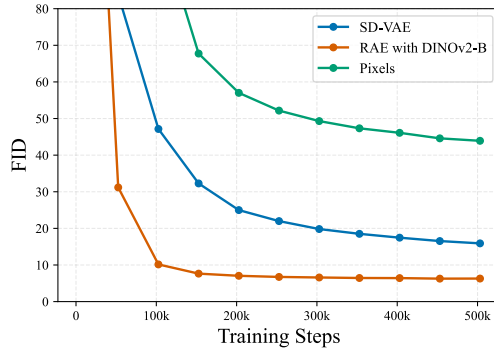
5.4.1. Training Efficiency

We evaluate the training efficiency in Fig. 5.9a and Fig. 5.9b by plotting the FID and IS metrics over training iterations. For a fair comparison, all representation spaces are evaluated without CFG guidance until convergence. We observe that RAEs, specifically when utilizing the DINOv2 latent space, significantly accelerate the training process. Leveraging these high-dimensional, semantically rich latent space enhances the learnability of the generation targets. Furthermore, the compressed SD-VAE space demonstrates superior learnability compared to the raw, high-dimensional pixel space, even when we utilize x-prediction for pixels. We further evaluate efficiency by scaling the FID over the total training computation measured in Giga Floating-Point Operations (GFLOPs) in Fig. 5.9c. This metric reveals a consistent trend, demonstrating that RAEs outpace both the SD-VAE and pixel-space representations in compute efficiency.

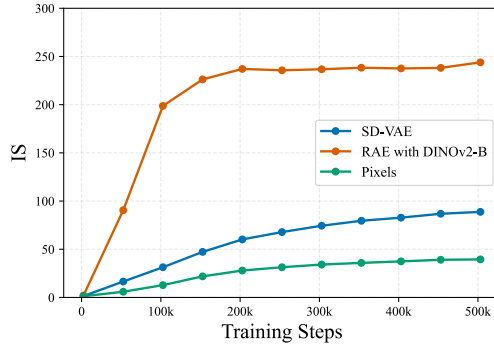
5.4.2. Final Generation Quality

In Tab. 5.3 we present the final generation results with and without CFG. A primary finding is that the choice of representation space has a strong influence to the CFG behavior. Crucially, the DINOv2 latent space does not benefit from guidance at all, achieving its optimal performance unguided (scale $w = 1.0$). While DINOv2 demonstrates high unguided generative fidelity, characterized by a high precision score (0.853), it simultaneously suffers from a significantly low recall score (0.301). This indicates a limitation in diversity whose underlying cause remains an open question for future study.

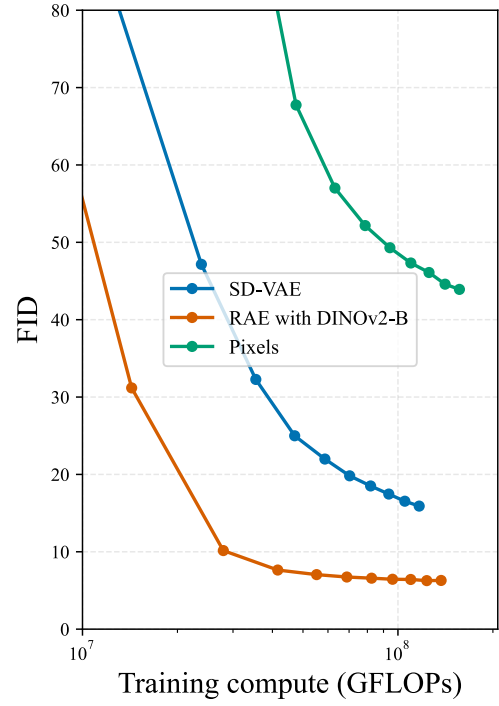
In contrast, SD-VAE and raw pixel space are heavily depend on guidance to reach peak performance. Without CFG, pixels demonstrate poor generation capabilities. We note that pixel space is not converged yet and still improves with further training, which highlights inefficient training. Finally, we note that while both the pixel space and RAEs benefit from a wider MLP denoiser block, the exact interplay between denoiser capacity and representation dimensionality is not yet fully explored.



(a) FID over training iterations.



(b) IS over training iterations.



(c) FID scaled with total training computation measured in GFLOPs.

Figure 5.9.: Comparison of training efficiency and convergence speed across different representation spaces. We compared all spaces until convergence without CFG guidance.

Rep. Space	Params.	Steps	CFG Scale	FID ↓	IS ↑	Prec. ↑	Rec. ↑
Without CFG							
SD-VAE (KL-16)	179.6M	503k	1.0	15.98	89.53	0.580	0.606
RAE (DINOv2-B)	165.4M	503k	1.0	6.43	247.63	0.853	0.301
Pixels	221.6M	503k	1.0	43.60	40.19	0.329	0.578
With CFG							
SD-VAE (KL-16)	179.6M	503k	6.0 (p)	3.19	263.84	0.757	0.507
RAE (DINOv2-B)	165.4M	503k	1.0 (-)	6.43	247.63	0.853	0.301
Pixels	221.6M	503k	2.0 (l)	8.70	216.76	0.688	0.346

Table 5.3.: Comparison of final generative performance across our representation spaces. All models are trained for 200 epochs (503k iterations). Indicators (p) and (l) denote progressive and linear CFG scheduling.

5.4.3. Masking during Training

To understand how different spaces process contextual information, we analyze the behavior of the transformer backbone under varying masking conditions. Specifically, after 200 epochs of training, we evaluate the trained models using an identical, fixed masking pattern applied across all three representation spaces under two distinct regimes: low masking (65%) and high masking (95%). We extract and visualize the global conditioning features z from the transformer backbone, which serve as the context for the per-token denoiser $p(x_i|z_i)$. In Fig. 5.10, the first row displays the PCA-colored features of the fully encoded image, while the second row illustrates the reconstructed features z learned by the transformer backbone during training.

Our observations reveal a shared characteristic across all representation spaces: a noticeable discrepancy between the unmasked ground-truth tokens and the reconstructed features. This effect is particularly visible in the low-masking regime (65%) (see Fig. 5.10), where much context is available. This phenomenon arises because gradients are exclusively updated for the masked tokens, as defined by the per-token loss in Eqn. 3.7. This is an architectural property originally inherited from MAE proposed by [4].

However, under the high-masking regime (95%), a clear divergence between the spaces emerges. The reconstructed features z for the pixel space appear considerably more blurry and structurally degraded. In contrast, the reconstruction within the DINOv2 latent space are remarkably clear and semantically coherent, even when 95% of the tokens are masked.

While high masking ratios closely simulate the initial steps of the autoregressive generation process (where very few tokens have been sampled), DINOv2 features provide an robust global context already from the first generated tokens.

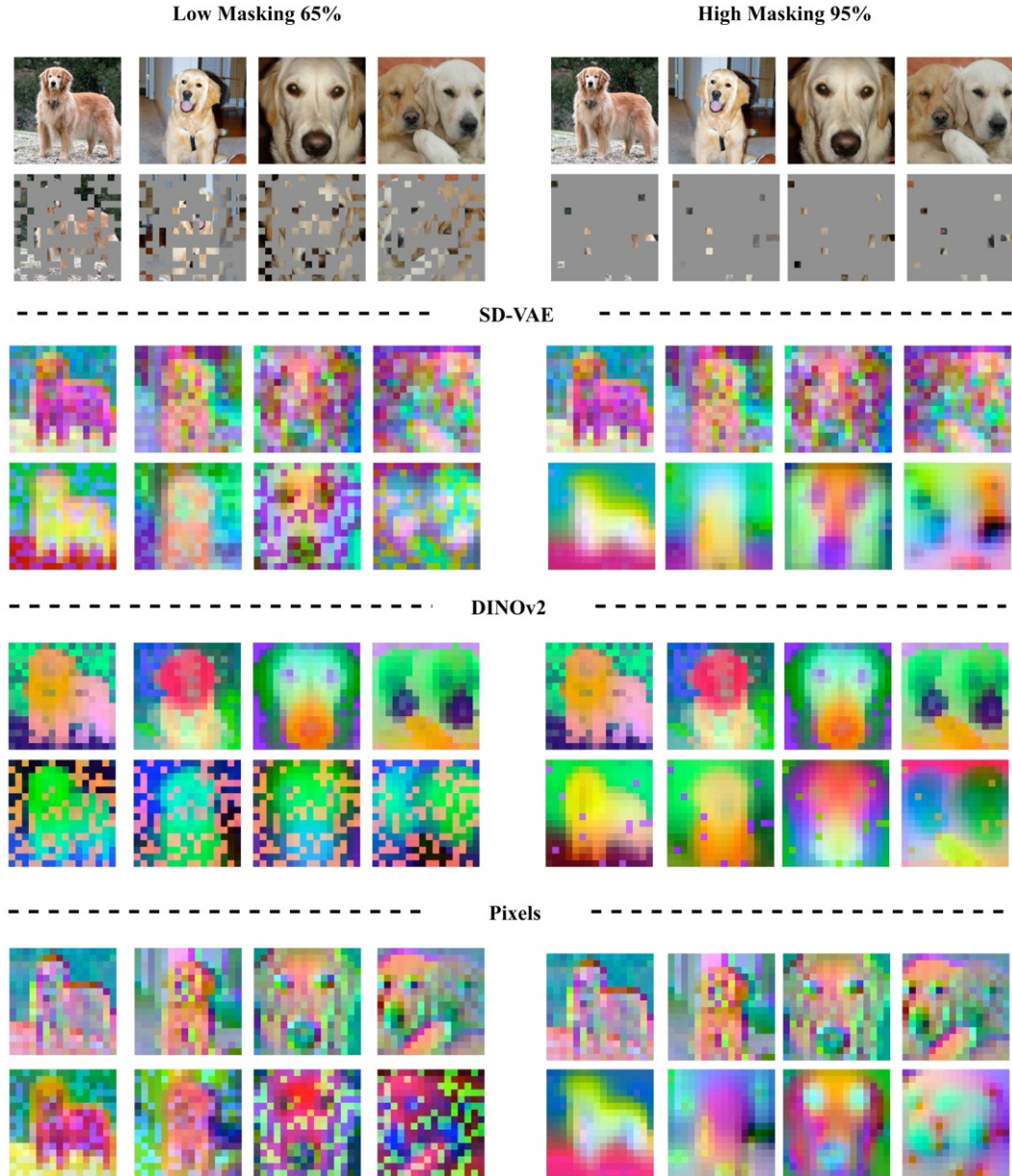


Figure 5.10.: Visual comparison of features under low (65%) and high (95%) masking ratios across different representation spaces. The first row visualizes the PCA-colored features of the **full encoded image**, while the second row displays the reconstructed **context features** z created by the transformer backbone. We note that no loss is computed on the visible (unmasked) tokens, therefore the transformer output on visible tokens in z is qualitatively worse. This is related to MAEs [4] and has no effect to the generation quality as we only denoise the masked tokens at inference.

5.4.4. Inference Parameters

Next, we evaluate the impact of different sampling configurations on generative performance.

Number of Autoregressive Steps. We begin by analyzing the number of autoregressive steps in Fig. 5.11. We observe a consistent trend across all three representation spaces. Specifically, the generation quality nearly converges at 32 steps, and even a highly accelerated schedule of just 8 steps yields acceptable visual results for DINOv2 and SD-VAE.

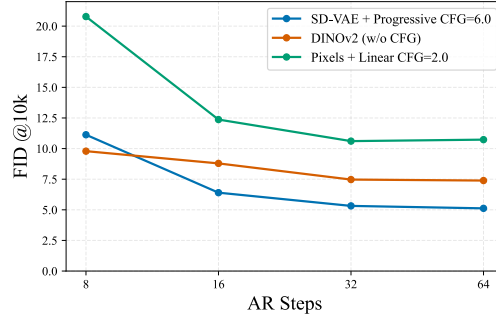
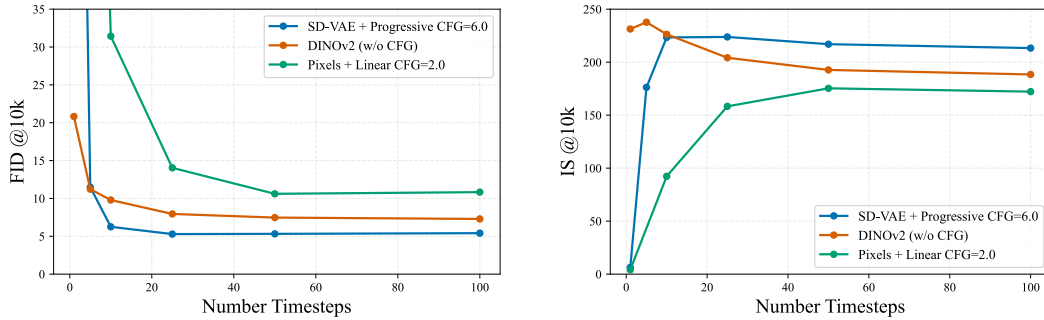


Figure 5.11.: FID over the total autoregressive steps across different representation spaces. Here, i.g., 32 steps mean we predict 8 token in a single step (for our sequence length of $N = 256$).



(a) FID@10k over different amount of timesteps t (b) IS@10k over different amount of timesteps t

Figure 5.12.: Comparing the generation quality over the number of timesteps during the ODE integration.

Number of Timesteps. Furthermore, we investigate the optimal number of ODE sampling steps required for inference. For all evaluations, we employ a first-order Euler integration scheme. Fig. 5.12 presents the FID and IS metrics evaluated across 10k samples as a function of the total ODE steps. We observe that the FID closely approaches convergence across all three representation spaces when using more

than 25 timesteps.

As reported in Tab. 5.4, an interesting finding emerges within the DINOv2 space, when the model synthesizes images with remarkably high perceptual fidelity using only a single generation step. Specifically, this single-step configuration achieves an FID = 20.82 and an IS = 231.31. In contrast, the SD-VAE baseline collapses entirely under a single-step regime, yielding a degraded FID = 138.81 and an IS = 6.30.

Repr. Space	FID@10k ↓	IS@10k ↑
SD-VAE (KL-16)	139.80	6.30
RAE (DINOv2-B)	20.82	231.31
Pixels	190.05	4.21

Table 5.4.: Comparing generation quality across representation spaces for a single timestep t with the identical loss formulation.

In Fig. 5.13, we provide a qualitative comparison of examples generated using a single timestep for both SD-VAE and DINOv2. While the DINOv2 model retains sharp structures and clear semantic definition, the SD-VAE baseline constructs only a coarse outline of the target object.

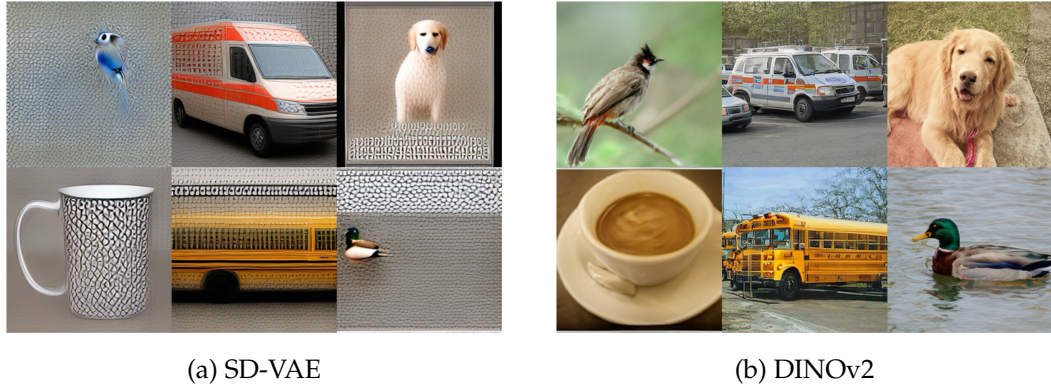


Figure 5.13.: Generated examples with a single timestep for SD-VAE and DINOv2. Surprisingly DINOv2 shows high fidelity generations while SD-VAE fails.

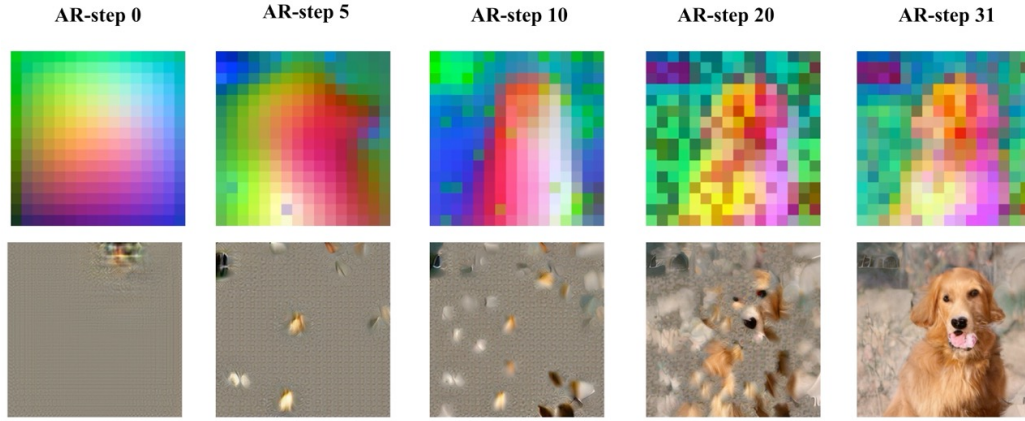
5.4.5. Visualization of the Autoregressive Generation Process

Finally, we examine the autoregressive generation process across. For this analysis, we select in total 32 autoregressive sampling steps. In Fig. 5.14, we plot the global context features z from the transformer backbone utilizing PCA coloring alongside the corresponding decoded images after steps 0, 5, 10, 20, and 31. Crucially, the decoded images are visualized after the per-token denoiser has cleaned the respective tokens.

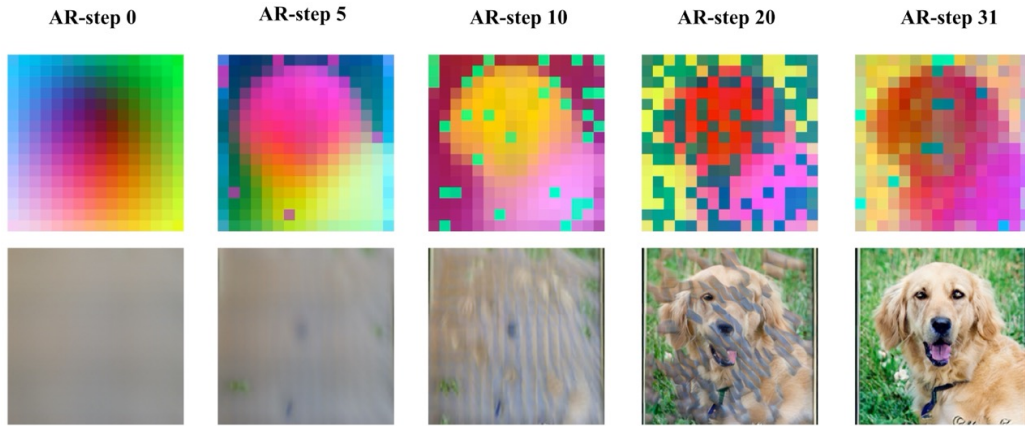
5.4. Comparison of Different Representation Spaces

As visualized in Fig. 5.14, we observe that across all representation spaces, the global features z exhibit structural instabilities as more denoised contexts are introduced in the later autoregressive steps. This mirrors the behavior previously observed in our training masking visualizations. But this does not effect the generation as we just use the masked tokens and ignore the generated context.

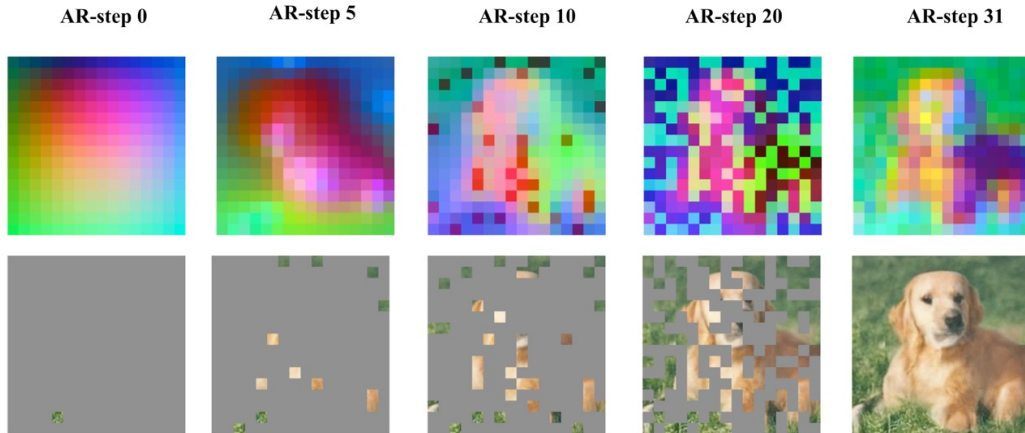
However, a key distinction is that DINOv2 establishes a stable and good semantic global context after only a few initial tokens have been sampled. Furthermore, analyzing the decoded intermediate images reveals a fundamental architectural contrast in token interaction. The DINOv2 latent space displays the strongest inter-token connectivity, where newly sampled tokens have the strongest influence for the contextual structure of neighboring patches. In contrast, the pixel space exhibits the lowest degree of inter-token influence by design.



(a) Autoregressive steps in SD-VAE latent space.



(b) Autoregressive steps in DINOv2 latent space.



(c) Autoregressive steps in pixel space.

Figure 5.14.: Visualization of the autoregressive generation process at step (0,5,10,20,and 31). For each representation space, the rows illustrate the PCA-colored global context features z and the corresponding decoded spatial outputs after the denoising.

6. Discussion

This chapter provides a final discussion of the research presented in this thesis. First, the key limitations of the study are examined, followed by an exploration of potential directions for future research. Finally, the thesis concludes with a summary of the core findings and overall contributions.

6.1. Limitations

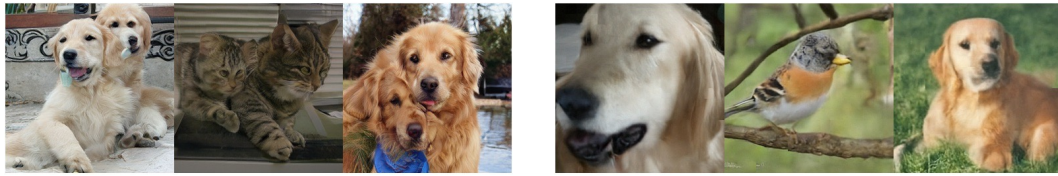
RAE Decoder Dependency. The generation quality within RAEs target space is fundamentally constrained by the performance of the pre-trained RAE decoder. In this study, we employed the pre-trained decoders introduced by [5]. Detailed specifications regarding this implementation are provided in A.1. Our evaluation of the reconstruction quality of RAE with DINOv2 encoder reveals that its reconstruction fidelity, as measured by LPIPS, is significantly lower than that of the baseline tokenizer SD-VAE. Furthermore, the decoders from [5] were trained with a noise term designed to reliably handle slightly off-manifold generations. We emphasize that this specific noise term may not align with the characteristics of the off-manifold generations produced by our masked generative models.

Optimal Hyperparameters and Architecture. Our findings indicate that mask-ratio schedule and timestep sampling distribution are inherently coupled to the topology of each individual representation space. For instance, in the DINOv2 latent space, a distribution shift toward high noise proved beneficial. In contrast, operating within the pixel space required a less aggressive mask-rate distribution during training. However, this exploration focused on evaluating existing configurations and established techniques rather than conducting a comprehensive hyperparameter optimization study. Furthermore, our results indicate a distinct capacity bottleneck in the denoiser when applied to high-dimensional manifolds. Specifically, increasing the MLP block ratio or the hidden dimension significantly improves generation quality.

Masking in RAE Target Space. A potential weakness of utilizing RAEs for masked flow-matching models is that the full image is first encoded into the representation space via the frozen RAE encoder before masking is applied. In a self-supervised space like DINOv2, an single token inherently encodes a larger spatial vicinity and richer semantic context. Consequently, it is easier for the transformer backbone to infer missing contextual information from the unmasked sequence than it is

when operating within the SD-VAE space. Due to this high informational density, a more aggressive masking schedule is required to prevent the prediction task from becoming trivial.

Failure Cases. For both the DINOv2 and pixel spaces, we empirically identified distinct failure modes in the generated images. While models operating in the DINOv2 target space exhibit remarkable generation fidelity, they frequently suffer from global structure and shape anomalies, as illustrated in Fig. 6.1a. For instance, the model fails to separate distinct semantic objects, such as co-occurring animals. In contrast, generations in the pixel space lack fine details and appear notably blurrier, as shown in Fig. 6.1b. We hypothesize that this degradation arises from undercapacity of the denoiser, which persists despite our use of x-prediction and a bottleneck layer ($d' = 128$).



(a) **DINOv2 space.** Demonstrates high perceptual fidelity but exhibits distinct failure modes in global coherence, particularly during interactions between multiple similar objects.

(b) **Pixel space.** Characterized by lack of fine details and a slight overall blurriness.

Figure 6.1.: Empirically observed failure cases for pixel and DINOv2 target space.

6.2. Further Research and Directions

Scalability. Due to computational resource constraints, our experimental evaluation was restricted to small architecture with parameters ranging from 164M to 221M. In recent literature, this corresponds to a Small (S) or Base (B) model configuration. In contrast, current SOTA research typically scales up to Large (L) and Huge (H) architectures. To provide a comparison against these models, scaling up our model is an essential next step. Consequently, exploring scalability behavior fell outside the scope of this work and remains open for future investigation. Additionally, our experiments were restricted to class-to-image generation on ImageNet-1k at a resolution of 256×256 . Extending this exploration to other conditional generative tasks, such as text-to-image synthesis at varied resolutions, is still required to compare masked generative models against joint diffusion and flow models.

Representation Alignment. Our study focused exclusively on evaluating masked generative models that utilize external and frozen autoencoders to define the target space. However, prior work [31, 33] demonstrates that aligning internal intermediate

features with vision foundation models also benefits the optimization of diffusion and flow models. Specifically, techniques like REPA can be employed to distill target representations into intermediate layers. Contrary to the prevalent assumption that RAE replaces the need for REPA, recent findings by [22] indicate that they exhibit complementary working mechanisms. Consequently, extending representation alignment methods to masked autoregressive models represents a highly promising direction for future research.

Guidance Strategy for DINOv2. During our experiments, we observed that CFG does not improve generation quality measured by the FID score. The similar observation was made by [5] for joint diffusion/flow models. To address this limitation, they implemented AutoGuidance [68], which leverages a weaker auxiliary model to guide the primary generative network. Alternatively, the recent work [22] demonstrated that REPA can be used as an implicit guidance mechanism. Because we did not explore alternative guidance strategies in our research, it remains an open field for masked generative models with DINOv2 latent space.

Auxiliary Losses. In this research, we exclusively employed the flow-matching loss to train both our transformer backbone and our per-token denoiser in parallel. Incorporating auxiliary structural or perceptual losses could enhance training efficiency and final fidelity. Such additions would likely be most relevant and impactful when operating directly in the pixel space.

6.3. Conclusion

In this thesis, we analyzed and compared distinct target representation spaces for masked autoregressive models utilizing continuous loss formulations. While previous foundational works in masked autoregressive modeling relied on DDPM formulations combined with stochastic samplers, recent research in generative modeling commonly built upon rectified flow. Motivated by these developments, we introduced a per-token loss function based on rectified flow, which leverages straight trajectories between the noise distribution and the target data.

Building on this formulation, we established a baseline within the widely utilized SD-VAE latent space. To ensure a fair comparison of across different representation spaces, we implemented a unified pipeline. All models were consistently trained for 200 epochs using identical batch sizes.

We observed that using a logit-normal distribution for the timestep sampling distribution yields benefits within the SD-VAE space. Furthermore, our exploration of various CFG schedules within this latent space indicated that a progressive scaling strategy achieves optimal performance. Consequently, we selected this configuration as our primary baseline for comparisons against alternative representation spaces.

Next, we systematically investigated key architectural parameters and configurations

within the DINOv2 target space. Our experiments revealed that a logit-normal timestep distribution with a shift toward higher noise levels is highly effective. Furthermore, we demonstrated that the complexity of the transformer backbone can be significantly reduced. A shallow transformer encoder suffices because the frozen DINOv2 features are already highly optimized for semantic decoding. We also determined that the capacity of the denoiser MLP plays a critical role when processing high-dimensional DINOv2 tokens. Finally, during our evaluation of various CFG schedules, we observed that guidance fails to improve performance within the DINOv2 space, even though the generated images already exhibit high fidelity.

Furthermore, we explored directly within the pixel space. The primary advantage of this approach is that it bypasses the need for a pre-trained tokenizer. We found that employing x -prediction and a bottleneck layer for the denoiser is beneficial, even though these components are not as strictly required as they are in joint diffusion/flow models. Similar to our observations with DINOv2, the capacity of the denoiser is highly relevant for processing the high-dimensional data. We determined that CFG is strictly necessary, unlike in the SD-VAE space, it must be applied starting from the first tokens. Ultimately, in terms of training efficiency, the pixel space proved to be the least practical target space, as it requires substantially more training beyond our 200-epoch training.

Given that training efficiency is very critical for generative models, combining a semantically meaningful target space with an efficiently learnable masked autoregressive flow formulation offers a highly viable approach. Our findings suggest that with further architectural optimizations and the integration of a suitable guidance strategy, such models are able to achieve a generation quality comparable to joint diffusion/flow models while remaining significantly more resource-efficient to train.

Bibliography

- [1] Robin Rombach, Andreas Blattmann, Dominik Lorenz, Patrick Esser, and Bjorn Ommer. High-Resolution Image Synthesis with Latent Diffusion Models. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2022.
- [2] Mathilde Caron, Hugo Touvron, Ishan Misra, Hervé Jégou, Julien Mairal, Piotr Bojanowski, and Armand Joulin. Emerging Properties in Self-Supervised Vision Transformers. In *IEEE/CVF International Conference on Computer Vision*, 2021.
- [3] Maxime Oquab, Timothée Darcet, Théo Moutakanni, Huy Vo, Marc Szafraniec, Vasil Khalidov, Pierre Fernandez, Daniel Haziza, Francisco Massa, Alaaeldin El-Nouby, Mahmoud Assran, Nicolas Ballas, Wojciech Galuba, Russell Howes, Po-Yao Huang, Shang-Wen Li, Ishan Misra, Michael Rabbat, Vasu Sharma, Gabriel Synnaeve, Hu Xu, Hervé Jégou, Julien Mairal, Patrick Labatut, Armand Joulin, and Piotr Bojanowski. DINOv2: Learning Robust Visual Features without Supervision, February 2024.
- [4] Kaiming He, Xinlei Chen, Saining Xie, Yanghao Li, Piotr Dollar, and Ross Girshick. Masked Autoencoders Are Scalable Vision Learners. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2022.
- [5] Boyang Zheng, Nanye Ma, Shengbang Tong, and Saining Xie. Diffusion Transformers with Representation Autoencoders. In *International Conference on Learning Representations*, 2026.
- [6] Tianhong Li and Kaiming He. Back to Basics: Let Denoising Generative Models Denoise, January 2026.
- [7] Tianhong Li, Yonglong Tian, He Li, Mingyang Deng, and Kaiming He. Autoregressive Image Generation without Vector Quantization. In *Advances in Neural Information Processing Systems*, 2024.
- [8] Zebin You, Jingyang Ou, Xiaolu Zhang, Jun Hu, Jun Zhou, and Chongxuan Li. Effective and Efficient Masked Image Generation Models. In *International Conference on Machine Learning*, 2025.
- [9] Patrick Esser, Sumith Kulal, Andreas Blattmann, Rahim Entezari, Jonas Müller, Harry Saini, Yam Levi, Dominik Lorenz, Axel Sauer, Frederic Boesel, Dustin Podell, Tim Dockhorn, Zion English, Kyle Lacey, Alex Goodwin, Yannik Marek, and Robin Rombach. Scaling Rectified Flow Transformers for High-Resolution Image Synthesis. In *International Conference on Machine Learning*, 2024.

Bibliography

- [10] Jascha Sohl-Dickstein, Eric A. Weiss, Niru Maheswaranathan, and Surya Ganguli. Deep Unsupervised Learning using Nonequilibrium Thermodynamics. In Francis R. Bach and David M. Blei, editors, *International Conference on Machine Learning*, 2015.
- [11] Jonathan Ho, Ajay Jain, and Pieter Abbeel. Denoising Diffusion Probabilistic Models. In *Advances in Neural Information Processing Systems*, 2020.
- [12] Jiaming Song, Chenlin Meng, and Stefano Ermon. Denoising Diffusion Implicit Models. In *International Conference on Learning Representations*, 2021.
- [13] Yang Song, Jascha Sohl-Dickstein, Diederik P. Kingma, Abhishek Kumar, Stefano Ermon, and Ben Poole. Score-Based Generative Modeling through Stochastic Differential Equations. In *International Conference on Learning Representations*. arXiv, 2021.
- [14] Yaron Lipman, Ricky T. Q. Chen, Heli Ben-Hamu, Maximilian Nickel, and Matt Le. Flow Matching for Generative Modeling. In *International Conference on Learning Representations*, 2023.
- [15] Xingchao Liu, Chengyue Gong, and Qiang Liu. Flow Straight and Fast: Learning to Generate and Transfer Data with Rectified Flow. In *International Conference on Learning Representations*. arXiv, 2023.
- [16] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention Is All You Need, August 2023.
- [17] Qiang Wang, Bei Li, Tong Xiao, Jingbo Zhu, Changliang Li, Derek F. Wong, and Lidia S. Chao. Learning Deep Transformer Models for Machine Translation, June 2019.
- [18] Alexey Dosovitskiy, Lucas Beyer, Alexander Kolesnikov, Dirk Weissenborn, Xiaohua Zhai, Thomas Unterthiner, Mostafa Dehghani, Matthias Minderer, Georg Heigold, Sylvain Gelly, Jakob Uszkoreit, and Neil Houlsby. An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale. In *International Conference on Learning Representations*, 2021.
- [19] William Peebles and Saining Xie. Scalable Diffusion Models with Transformers. In *IEEE/CVF International Conference on Computer Vision*, 2023.
- [20] Nanye Ma, Mark Goldstein, Michael S. Albergo, Nicholas M. Boffi, Eric Vandenberg, and Saining Xie. SiT: Exploring Flow and Diffusion-Based Generative Models with Scalable Interpolant Transformers. In Ales Leonardis, Elisa Ricci, Stefan Roth, Olga Russakovsky, Torsten Sattler, and Gül Varol, editors, *European Conference on Computer Vision*, 2024.
- [21] Black Forest Labs, Stephen Batifol, Andreas Blattmann, Frederic Boesel, Saksham Consul, Cyril Diagne, Tim Dockhorn, Jack English, Zion English, Patrick Esser,

- Sumith Kulal, Kyle Lacey, Yam Levi, Cheng Li, Dominik Lorenz, Jonas Müller, Dustin Podell, Robin Rombach, Harry Saini, Axel Sauer, and Luke Smith. FLUX.1 Kontext: Flow Matching for In-Context Image Generation and Editing in Latent Space, June 2025.
- [22] Jaskirat Singh, Boyang Zheng, Zongze Wu, Richard Zhang, Eli Shechtman, and Saining Xie. Improved Baselines with Representation Autoencoders, May 2026.
- [23] Alan Baade, Eric Ryan Chan, Kyle Sargent, Changan Chen, Justin Johnson, Ehsan Adeli, and Li Fei-Fei. Latent Forcing: Reordering the Diffusion Trajectory for Pixel-Space Image Generation, February 2026.
- [24] Yongsheng Yu, Wei Xiong, Weili Nie, Yichen Sheng, Shiqiu Liu, and Jiebo Luo. PixelDiT: Pixel Diffusion Transformers for Image Generation, November 2025.
- [25] Yiyang Lu, Susie Lu, Qiao Sun, Hanhong Zhao, Zhicheng Jiang, Xianbang Wang, Tianhong Li, Zhengyang Geng, and Kaiming He. One-step Latent-free Image Generation with Pixel Mean Flows, February 2026.
- [26] Ting Chen, Simon Kornblith, Mohammad Norouzi, and Geoffrey Hinton. A Simple Framework for Contrastive Learning of Visual Representations. In *International Conference on Machine Learning*, 2020.
- [27] Kaiming He, Haoqi Fan, Yuxin Wu, Saining Xie, and Ross Girshick. Momentum Contrast for Unsupervised Visual Representation Learning. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2020.
- [28] Oriane Siméoni, Huy V. Vo, Maximilian Seitzer, Federico Baldassarre, Maxime Oquab, Cijo Jose, Vasil Khalidov, Marc Szafraniec, Seungeun Yi, Michaël Ramamonjisoa, Francisco Massa, Daniel Haziza, Luca Wehrstedt, Jianyuan Wang, Timothée Darcet, Théo Moutakanni, Leonel Sentana, Claire Roberts, Andrea Vedaldi, Jamie Tolan, John Brandt, Camille Couprie, Julien Mairal, Hervé Jégou, Patrick Labatut, and Piotr Bojanowski. DINOv3, August 2025.
- [29] Alec Radford, Jong Wook Kim, Chris Hallacy, Aditya Ramesh, Gabriel Goh, Sandhini Agarwal, Girish Sastry, Amanda Askell, Pamela Mishkin, Jack Clark, Gretchen Krueger, and Ilya Sutskever. Learning Transferable Visual Models From Natural Language Supervision. In *International Conference on Machine Learning*, 2021.
- [30] Xiaohua Zhai, Basil Mustafa, Alexander Kolesnikov, and Lucas Beyer. Sigmoid Loss for Language Image Pre-Training. In *IEEE/CVF International Conference on Computer Vision*. arXiv, 2023.
- [31] Sihyun Yu, Sangkyung Kwak, Huiwon Jang, Jongheon Jeong, Jonathan Huang, Jinwoo Shin, and Saining Xie. Representation Alignment for Generation: Training Diffusion Transformers Is Easier Than You Think. In *International Conference on Learning Representations*, 2025.

Bibliography

- [32] Hila Chefer, Patrick Esser, Dominik Lorenz, Dustin Podell, Vikash Raja, Vinh Tong, Antonio Torralba, and Robin Rombach. Self-Supervised Flow Matching for Scalable Multi-Modal Synthesis, March 2026.
- [33] Tianhong Li, Dina Katabi, and Kaiming He. Return of Unconditional Generation: A Self-supervised Representation Generation Method. In *Advances in Neural Information Processing Systems*, 2024.
- [34] Black Forest Labs. Representation Comparison: Analyzing and Enhancing the Latent Space of FLUX. <https://bfl.ai/research/representation-comparison>, January 2025.
- [35] Jingfeng Yao, Bin Yang, and Xinggang Wang. Reconstruction vs. Generation: Taming Optimization Dilemma in Latent Diffusion Models. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2025.
- [36] Aaron van den Oord, Nal Kalchbrenner, Oriol Vinyals, Lasse Espeholt, Alex Graves, and Koray Kavukcuoglu. Conditional Image Generation with PixelCNN Decoders. In *Advances in Neural Information Processing Systems*, 2016.
- [37] Mark Chen, Alec Radford, Rewon Child, Jeffrey Wu, Heewoo Jun, David Luan, and Ilya Sutskever. Generative Pretraining From Pixels. In *International Conference on Machine Learning*, 2020.
- [38] Patrick Esser, Robin Rombach, and Bjorn Ommer. Taming Transformers for High-Resolution Image Synthesis. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2021.
- [39] Aditya Ramesh, Mikhail Pavlov, Gabriel Goh, Scott Gray, Chelsea Voss, Alec Radford, Mark Chen, and Ilya Sutskever. Zero-shot text-to-image generation. *CoRR*, abs/2102.12092, 2021.
- [40] Peize Sun, Yi Jiang, Shoufa Chen, Shilong Zhang, Bingyue Peng, Ping Luo, and Zehuan Yuan. Autoregressive Model Beats Diffusion: Llama for Scalable Image Generation. In *Conference on Computer Vision and Pattern Recognition*, 2024.
- [41] Keyu Tian, Yi Jiang, Zehuan Yuan, Bingyue Peng, and Liwei Wang. Visual Autoregressive Modeling: Scalable Image Generation via Next-Scale Prediction. In Amir Globersons, Lester Mackey, Danielle Belgrave, Angela Fan, Ulrich Paquet, Jakub M. Tomczak, and Cheng Zhang, editors, *Advances in Neural Information Processing Systems*, 2024.
- [42] Huiwen Chang, Han Zhang, Lu Jiang, Ce Liu, and William T. Freeman. MaskGIT: Masked Generative Image Transformer. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 11305–11315, 2022.
- [43] Tianhong Li, Huiwen Chang, Shlok Kumar Mishra, Han Zhang, Dina Katabi, and Dilip Krishnan. MAGE: MAsked Generative Encoder to Unify Representation

- Learning and Image Synthesis. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2023.
- [44] Lijie Fan, Tianhong Li, Siyang Qin, Yuanzhen Li, Chen Sun, Michael Rubinstein, Deqing Sun, Kaiming He, and Yonglong Tian. Fluid: Scaling Autoregressive Text-to-image Generative Models with Continuous Tokens. In *International Conference on Learning Representations*, 2025.
 - [45] Sucheng Ren, Qihang Yu, Ju He, Xiaohui Shen, Alan Yuille, and Liang-Chieh Chen. Beyond Next-Token: Next-X Prediction for Autoregressive Visual Generation. In *IEEE/CVF International Conference on Computer Vision*, 2025.
 - [46] Tim Salimans, Ian Goodfellow, Wojciech Zaremba, Vicki Cheung, Alec Radford, and Xi Chen. Improved Techniques for Training GANs. In *Advances in Neural Information Processing Systems*, 2016.
 - [47] Christian Szegedy, Vincent Vanhoucke, Sergey Ioffe, Jonathon Shlens, and Zbigniew Wojna. Rethinking the Inception Architecture for Computer Vision. In *IEEE Conference on Computer Vision and Pattern Recognition*, 2016.
 - [48] Martin Heusel, Hubert Ramsauer, Thomas Unterthiner, Bernhard Nessler, and Sepp Hochreiter. GANs Trained by a Two Time-Scale Update Rule Converge to a Local Nash Equilibrium, January 2018.
 - [49] Tuomas Kynkäänniemi, Tero Karras, Samuli Laine, Jaakko Lehtinen, and Timo Aila. Improved Precision and Recall Metric for Assessing Generative Models. In *Advances in Neural Information Processing Systems*, 2019.
 - [50] Richard Zhang, Phillip Isola, Alexei A. Efros, Eli Shechtman, and Oliver Wang. The Unreasonable Effectiveness of Deep Features as a Perceptual Metric. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2018.
 - [51] Karen Simonyan and Andrew Zisserman. Very Deep Convolutional Networks for Large-Scale Image Recognition. In *International Conference on Learning Representations*, 2015.
 - [52] Alex Krizhevsky, Ilya Sutskever, and Geoffrey E Hinton. ImageNet Classification with Deep Convolutional Neural Networks. In *Advances in Neural Information Processing Systems*, 2012.
 - [53] Forrest N. Iandola, Song Han, Matthew W. Moskewicz, Khalid Ashraf, William J. Dally, and Kurt Keutzer. SqueezeNet: AlexNet-level accuracy with 50x fewer parameters and <0.5MB model size. In *International Conference on Learning Representations*, 2017.
 - [54] Emiel Hoogeboom, Jonathan Heek, and Tim Salimans. Simple diffusion: End-to-end diffusion for high resolution images. In *International Conference on Machine Learning*, 2023.

Bibliography

- [55] J. Aitchison and S. M. Shen. Logistic-Normal Distributions: Some Properties and Uses. *Biometrika*, 1980.
- [56] Jonathan Ho and Tim Salimans. Classifier-Free Diffusion Guidance, July 2022.
- [57] Michael S. Albergo and Eric Vanden-Eijnden. Building Normalizing Flows with Stochastic Interpolants. In *International Conference on Learning Representations*, March 2023.
- [58] Laurens van der Maaten and Geoffrey Hinton. Visualizing Data using t-SNE. *Journal of Machine Learning Research*, 2008.
- [59] T. Tony Cai and Rong Ma. Theoretical Foundations of t-SNE for Visualizing High-Dimensional Clustered Data, November 2022.
- [60] Noam Shazeer. GLU Variants Improve Transformer, February 2020.
- [61] Biao Zhang and Rico Sennrich. Root Mean Square Layer Normalization. In *Advances in Neural Information Processing Systems*, 2019.
- [62] Jianlin Su, Yu Lu, Shengfeng Pan, Ahmed Murtadha, Bo Wen, and Yunfeng Liu. RoFormer: Enhanced Transformer with Rotary Position Embedding, November 2023.
- [63] Ethan Perez, Florian Strub, Harm de Vries, Vincent Dumoulin, and Aaron Courville. FiLM: Visual Reasoning with a General Conditioning Layer, December 2017.
- [64] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep Residual Learning for Image Recognition. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2016.
- [65] Olga Russakovsky, Jia Deng, Hao Su, Jonathan Krause, Sanjeev Satheesh, Sean Ma, Zhiheng Huang, Andrej Karpathy, Aditya Khosla, Michael Bernstein, Alexander C. Berg, and Li Fei-Fei. ImageNet Large Scale Visual Recognition Challenge, January 2015.
- [66] Anton Obukhov, Maximilian Seitzer, Po-Wei Wu, Semen Zhydenko, Jonathan Kyl, and Elvis Yu-Jing Lin. High-fidelity performance metrics for generative models in pytorch, 2020. Version: 0.3.0, DOI: 10.5281/zenodo.4957738.
- [67] Prafulla Dhariwal and Alexander Nichol. Diffusion Models Beat GANs on Image Synthesis. In *Advances in Neural Information Processing Systems*, 2021.
- [68] Tero Karras, Miika Aittala, Tuomas Kynkäänniemi, Jaakko Lehtinen, Timo Aila, and Samuli Laine. Guiding a Diffusion Model with a Bad Version of Itself. In *Advances in Neural Information Processing Systems*, 2024.
- [69] Axel Sauer, Tero Karras, Samuli Laine, Andreas Geiger, and Timo Aila. StyleGAN-T: Unlocking the Power of GANs for Fast Large-Scale Text-to-Image Synthesis. In *International Conference on Machine Learning*, 2023.

- [70] Ian J. Goodfellow, Jean Pouget-Abadie, Mehdi Mirza, Bing Xu, David Warde-Farley, Sherjil Ozair, Aaron Courville, and Yoshua Bengio. Generative Adversarial Networks. In *Advances in Neural Information Processing Systems*, 2014.

A. Additional Implementation Details

A.1. Details for Representation Autoencoders (RAE)

Encoder Details. For all frozen encoders we systematically discard the global class token embedding (CLS), restricting the representation space exclusively to the sequence of patch tokens. All encoders operate on a standard Vision Transformer (ViT) architecture utilizing full bidirectional self-attention mechanisms. We applied a channel-wise Layer Normalization to ensure that each token embedding possesses a zero mean and unit variance across its hidden channels.

The DINOv2 and MAE foundational encoders provide weights pre-trained with an explicit patch size of $p_e = 14$ with a image resolution of 224×224 pixels. To maintain structural consistency across our pipeline, we follow [5] and apply bilinear interpolation to the intermediate tokens, scaling them up to a uniform target input resolution of 256×256 pixels. This interpolation increases the spatial token distribution to yield a standardized layout containing exactly 256 functional spatial context tokens.

Decoder Details. We use the pre-trained decoders from [5]. They are trained on the ImageNet-1k at a fixed spatial resolution of 256×256 pixels. The primary optimization objective is to learn a stable, invertible mapping that projects the high-dimensional token embeddings generated by the frozen foundational encoders back into raw image pixels.

Architecturally, the decoding pipeline adapts the structural design of StyleGAN-T [69]. To guide the adversarial feedback loops, it utilizes a frozen DINO-S/8 model [2] acting as a fixed semantic discriminator. The specific optimization hyperparameters governing the decoder training routines are systematically itemized in Tab. A.1.

To achieve high structural fidelity during reconstruction, the model optimizes a loss objective combining an ℓ_1 structural loss, perceptual loss via the LPIPS metric [50], and an auxiliary adversarial loss [70]. The unified reconstruction loss function is formulated with:

$$\mathcal{L}_{\text{rec}} = w_L \cdot \mathcal{L}_{\text{LPIPS}}(\hat{x}, x) + w_G \cdot \lambda \cdot \mathcal{L}_{\text{GAN}}(\hat{x}, x) + \ell_1, \quad (\text{A.1})$$

where $x \in \mathbb{R}^{3 \times H \times W}$ represents the target ground-truth pixel image and $\hat{x} = D(E(x))$ defines the reconstructed output synthesized by the projection decoder D .

Component	Decoder	Discriminator
optimizer	Adam	Adam
max learning rate	2×10^{-4}	2×10^{-4}
min learning rate	2×10^{-5}	2×10^{-5}
learning rate schedule	cosine decay	cosine decay
optimizer betas	(0.5, 0.9)	(0.5, 0.9)
weight decay	0.0	0.0
batch size	512	512
warmup	1 epoch	1 epoch
loss	$\ell_1 + \text{LPIPS} + \text{GAN}$	adv.
Model	ViT-(B, L, XL)	Dino-S/8 (frozen)
LPIPS start epoch	0	–
disc. start epoch	–	6
adv. loss start epoch	8	–
Training epochs	16	10

Table A.1.: Training details of the RAE decoders from [5].

A.2. Detailed Parameters and Architectures

In Tab. A.2, we present the architectural, training and sampling configurations used for our masked autoregressive flow matching models for all representation spaces. The Table is structurally categorized into neural network architectures, training optimization details, and specific inference sampling details.

A.2. Detailed Parameters and Architectures

Repr. Space	FlowMAR-B		
	SD-VAE	DINOv2-B	Pixels
architecture			
parameters (M)	179.6M	165.4M	221.6M
AR-trans. encoder depth	10	2	10
AR-trans. decoder depth	10	10	10
AR-trans. hidden dim	768	768	768
AR-trans. heads	12	12	12
AR-trans. architecture	MAE-style	MAE-style	MAE-style
AR-trans. dropout	0.1	0.1	0.1
AR-trans. class tokens	64	64	64
denoiser depth	6	6	6
denoiser hidden dim	1024	1024	1024
denoiser block ratio	1	4	4
denoiser dropout	0.0	0.0	0.0
image size	256	256	256
patch size	1	1	image / 16
bottleneck	-	-	128
training			
epochs	200	200	200
warmup epochs	50	50	50
optimizer	Adam, $\beta_1, \beta_2 = 0.9, 0.95$	Adam, $\beta_1, \beta_2 = 0.9, 0.95$	Adam, $\beta_1, \beta_2 = 0.9, 0.95$
batch size	512	512	512
learning rate	2e-4	2e-4	2e-4
learning rate schedule	constant	constant	constant
weight decay	0.02	0.02	0.02
ema decay	0.9999	0.9999	0.9999
time sampler	logit-normal	logit-normal + shift	logit-normal
mask rate sampler	Exp + clamb 0.62	Exp + clamb 0.62	trunc normal 0.7
class token drop (for CFG)	0.1	0.1	0.1
sampling			
ODE solver	Euler	Euler	Euler
ODE steps	50	50	50
time steps	linear in [0.0, 1.0]	shifted in [0.0, 1.0]	linear in [0.0, 1.0]
mask schedule	cosine	cosine	cosine
AR steps	32	32	32
generation order	random	random	random

Table A.2.: Final implementation details for our flowMAR-B model and all representation spaces.

B. Qualitative Results

In Fig. B.1, B.2 and B.3, we provide additional generated examples of all three representation spaces trained on ImageNet 256×256 .

Appendix B. Qualitative Results



(a) class 088: Parrot



(b) class 107: Jellyfish



(c) class 128: White stork



(d) class 279: Artic fox



(e) class 281: Tabby cat



(f) class 417: Balloon



(g) class 425: Barn



(h) class 930: French loaf

Figure B.1.: Additional samples generated within the SD-VAE latent space.



(a) class 088: Parrot



(b) class 107: Jellyfish



(c) class 128: White stork



(d) class 279: Artic fox



(e) class 281: Tabby cat



(f) class 417: Balloon



(g) class 425: Barn



(h) class 930: French loaf

Figure B.2.: Additional samples generated within the DINOv2 latent space.

Appendix B. Qualitative Results



(a) class 088: Parrot



(b) class 107: Jellyfish



(c) class 128: White stork



(d) class 279: Artic fox



(e) class 281: Tabby cat



(f) class 417: Balloon



(g) class 425: Barn



(h) class 930: French loaf

Figure B.3.: Additional samples generated within the pixel space.

Erklärung

Laut Beschlüssen der Prüfungsausschüsse Bioinformatik, Informatik, Informatik Lehramt, Kognitionswissenschaft, Machine Learning, Medieninformatik und Medizininformatik der Universität Tübingen vom 05.02.2025. Gültig für Abschlussarbeiten (B.Sc./M.Sc./B.Ed./M.Ed.) in den zugehörigen Fächern. Bei Studienarbeiten und Hausarbeiten bitte nach Maßgabe des/der jeweiligen Prüfers/Prüferin.

1. Allgemeine Erklärungen

Hiermit erkläre ich:

- Ich habe die vorgelegte Arbeit selbständig verfasst und keine anderen als die angegebenen Quellen und Hilfsmittel benutzt.
- Ich habe alle wörtlich oder sinngemäß aus anderen Werken übernommenen Aussagen als solche gekennzeichnet.
- Die Arbeit war weder vollständig noch in wesentlichen Teilen Gegenstand eines anderen Prüfungsverfahrens.
- Falls ich ein elektronisches Exemplar und eines oder mehrere gedruckte und gebundene Exemplare eingereicht habe (z.B., weil der/die Prüfer/in(nen) dies wünschen): Das elektronisch eingereichte Exemplar stimmt exakt mit dem bzw. den von mir eingereichten gedruckten und gebundenen Exemplar(en) überein.

2. Erklärung bezüglich Veröffentlichungen

Eine Veröffentlichung ist häufig ein Qualitätsmerkmal (z.B. bei Veröffentlichung in Fachzeitschrift, Konferenz, Preprint, etc.). Sie muss aber korrekt angegeben werden. Bitte kreuzen Sie die für Ihre Arbeit zutreffende Variante an:

- ☒ Die Arbeit wurde bisher weder vollständig noch in Teilen veröffentlicht.
- ☐ Die Arbeit wurde in Teilen oder vollständig schon veröffentlicht. Hierfür findet sich im Anhang eine vollständige Tabelle mit bibliographischen Angaben.

3. Nutzung von Methoden der künstlichen Intelligenz (KI, z.B. chatGPT, DeepL, etc.)

Die Nutzung von KI kann sinnvoll sein. Sie muss aber korrekt angegeben werden und kann die Schwerpunkte bei der Bewertung der Arbeit beeinflussen. Bitte kreuzen Sie alle für Ihre Arbeit zutreffenden Varianten an und beachten Sie, dass die Varianten 3.4 - 3.6 eine vorherige Absprache mit dem/der Betreuer/in voraussetzen:

- ☐ 3.1. Keine Nutzung: Ich habe zur Erstellung meiner Arbeit keine KI benutzt.
- ☒ 3.2. Korrektur Rechtschreibung & Grammatik: Ich habe KI für Korrekturen der Rechtschreibung und Grammatik genutzt, ohne dass es dabei zu inhaltlich relevanter Textgeneration oder Übersetzungen kam. Das heißt, ich habe von mir verfasste Texte in derselben Sprache korrigieren lassen. Es handelt sich um rein sprachliche Korrekturen, sodass die von mir ursprünglich intendierte Bedeutung nicht wesentlich verändert oder erweitert wurde. Im Zweifelsfall habe ich mich mit meinem/r Betreuer/in besprochen. Alle genutzten Programme mit Versionsnummer sind im Anhang meiner Arbeit in einer Tabelle aufgelistet.

- ☒ 3.3. Unterstützung bei der Softwareentwicklung: Ich habe KI als Unterstützung beim Schreiben von Code in der Softwareentwicklung genutzt. Es handelt sich hierbei lediglich um Unterstützung und nicht um die automatische Generierung von größeren Programm-Teilen. Im Zweifelsfall habe ich mich mit meinem/r Betreuer/in besprochen. Alle genutzten Programme mit Versionsnummer sind im Anhang meiner Arbeit in einer Tabelle aufgelistet.
- ☐ 3.4. Übersetzung: Ich habe *nach vorheriger Absprache und mit Erlaubnis meines/r Betreuer/in* KI zur Übersetzung von mir in einer anderen Sprache geschriebenen Texte genutzt. Jede derartige Übersetzung ist im laufenden Text gekennzeichnet und der Anhang meiner Arbeit enthält eine Tabelle mit einem vollständigen Nachweis aller übersetzten Textstellen und der verwendeten Programme mit Versionsnummer.
- ☐ 3.5. Code-Generierung: Ich habe *nach vorheriger Absprache und mit Erlaubnis meines/r Betreuer/in* KI zur Erzeugung von Code in der Softwareentwicklung genutzt. Der Anhang meiner Arbeit enthält eine Tabelle mit einem vollständigen Nachweis aller derartigen Nutzungen, der verwendeten Programme mit Versionsnummer und der verwendeten Prompts.
- ☐ 3.6. Text-Generierung: Ich habe *nach vorheriger Absprache und mit Erlaubnis meines/r Betreuer/in* KI zur Erzeugung von Text in meiner Arbeit genutzt. Jede derartige Verwendung von KI ist im laufenden Text gekennzeichnet und der Anhang meiner Arbeit enthält eine Tabelle mit einem vollständigen Nachweis aller derartigen Nutzungen, der verwendeten Programme mit Versionsnummer und der verwendeten Prompts.

Falls ich in irgendeiner Form KI genutzt haben (siehe oben), dann erkläre ich:

Mir ist bewusst, dass ich die Verantwortung trage, falls es durch die Verwendung von KI zu fehlerhaften Inhalten, zu Verstößen gegen das Datenschutzrecht, Urheberrecht oder zu wissenschaftlichem Fehlverhalten (z.B. Plagiaten) kommt.

4. Abschluss und Unterschrift(en)

Mir ist bekannt, dass ein Verstoß gegen diese Erklärung prüfungsrechtliche Konsequenzen haben und insbesondere dazu führen kann, dass die Prüfungsleistung mit „nicht ausreichend“ bzw. die Studienleistung mit „nicht bestanden“ bewertet wird und bei mehrfachem oder schwerwiegendem Täuschungsversuch eine Exmatrikulation erfolgen bzw. ein Verfahren zur Entziehung eines eventuell verliehenen akademischen Titels eingeleitet werden kann.

<u>Marcel Plocher</u>	<u>Tübingen, 30.05.26</u>	<u>M. Plocher</u>
Vorname, Nachname Student/in	Ort, Datum	Unterschrift

Die Punkte 3.4 - 3.6 erfordern eine Zustimmung des/r Betreuer/in. Sollten Sie einen dieser Punkte angekreuzt haben, dann sollte der/die Betreuer/in bitte hier unterschreiben:

Ich habe der oben genannten Nutzung von KI zur Erstellung der Arbeit zugestimmt.

_____	_____	_____
Vorname, Nachname Betreuer/in	Ort, Datum	Unterschrift

AI Tool Usage

AI-System	Provider	Version	Usage
Gemini	Google	latest	Text revision
DeepL Write	DeepL SE	latest	Text revision
Claude	Anthropic	Sonnet 4.6	Coding, Debugging

Table B.1.: Used AI tool in context of this thesis